

Linux File Hierarchy Structure (FHS) & File Types — Study Notes

1. What is FHS?

FHS = Filesystem Hierarchy Standard. It defines a standard layout for directories and their contents on Linux, so software and users know where to expect things (configs in one place, logs in another, etc.).

Everything starts from the **root directory** `/` — all other directories branch out from it.

2. Core FHS Directories

`/` — Root Directory

- Topmost directory; everything else lives under it.
 - Only **root user** can normally modify files directly inside `/`.
-

`/etc` — Configuration Files

System-wide config files for users, network, services, and applications.

```
/etc/passwd      # user account info
/etc/resolv.conf  # DNS resolver config
/etc/logrotate.conf
```

`/home` — User Home Directories

Personal directories for **normal (non-root) users** — Documents, Downloads, personal settings.

```
/home/veenay/Documents
```

`/var` — Variable Data

Contains data that **changes/grows constantly** while the system runs — the opposite of static config files.

Subdirectory	Contains
<code>/var/log</code>	System & application log files
<code>/var/spool</code>	Print/mail queues, cron jobs waiting to run

Subdirectory	Contains
<code>/var/cache</code>	Cached data from applications
<code>/var/www</code>	Web server content (common on many distros)
<code>/var/lib</code>	Persistent application state/data (e.g., databases)

```
tail -f /var/log/syslog      # very common real-world command
```

● **Exam Trap:** `/etc` = static configuration (rarely changes), `/var` = dynamic/growing data (changes constantly).

`/proc` — Process & System Information

A **virtual/pseudo filesystem** — not real files stored on disk. It's generated live by the kernel to expose info about running processes, CPU, memory, uptime, etc.

```
/proc/1234      # info for process with PID 1234
/proc/meminfo   # memory info
/proc/uptime    # system uptime
/proc/cpuinfo   # CPU details
```

`/tmp` — Temporary Files

Temporary files created by programs/users. May be **auto-cleared** on reboot.

● **Note:** `/tmp` is world-writable by design, but protected by the **Sticky Bit** — only the file's owner (or root) can delete/rename files inside it, even though anyone can create files there.

`/usr` — User Programs & Utilities

Holds the bulk of installed programs, libraries, and documentation (despite the name, NOT user home data — that's `/home`).

Subdirectory	Contains
<code>/usr/bin</code>	Most user commands (<code>awk</code> , <code>less</code> , <code>scp</code>)
<code>/usr/sbin</code>	Admin commands (<code>sshd</code> , <code>cron</code> , <code>useradd</code>)
<code>/usr/lib</code>	Libraries used by <code>/usr/bin</code> & <code>/usr/sbin</code> programs
<code>/usr/local</code>	Software installed manually/from source (not via package manager)
<code>/usr/src</code>	Kernel source code, headers

● **Exam Trap:** Don't confuse `/usr` (programs) with `/home` (personal user files).

`/bin` — Essential User Commands

Basic commands needed by **all users**, available even in minimal/recovery mode.

```
ls, cp, ping, grep, ps, kill
```

💡 On modern distros, `/bin` is often just a **symlink** to `/usr/bin` (merged-usr layout).

3. Other Important FHS Directories

`/boot` — Boot Files

Files needed to **start Linux** — kernel, bootloader, initial RAM disk.

```
vmlinux      # Linux kernel image
initrd.img   # Initial RAM disk
grub/        # GRUB bootloader files
```

`/dev` — Device Files

Linux treats hardware as files. `/dev` holds these device representations.

```
/dev/sda      # first hard disk
/dev/sda1     # first partition on that disk
/dev/tty1     # terminal
```

`/lib` — Shared Libraries

Libraries required by programs in `/bin` and `/sbin` to actually run.

```
libcurses.so
ld-2.11.1.so
```

`/media` — Removable Media

Auto-mount point for **removable devices**: USB, CD/DVD, pen drives.

`/mnt` — Temporary Manual Mount Point

Used by **admins** to manually/temporarily mount a filesystem.

```
mount /dev/sdb1 /mnt
```

/media vs /mnt

Feature	/media	/mnt
Used for	Removable devices (auto-mount)	Manual/temporary mounts by admin
Who mounts	System (automatically)	Sysadmin (manually)

/opt — Optional / Third-Party Software

Software **not part of the default OS install** — usually vendor apps.

```
/opt/company-name/application
```

/sbin — System Administration Commands

Commands mainly used by **root/sysadmins** for system maintenance.

```
fdisk, reboot, iptables, fsck, swapon
```

/bin vs /sbin

Directory	Used By	Examples
/bin	All users	ls, cp, grep
/sbin	Admins/root	fdisk, reboot, fsck

/srv — Service Data

Data served by services running on the system (web, FTP, version control).

```
/srv/cvs  
/srv/www
```

/usr/local — Manually Installed Software

Part of `/usr`, but called out separately because it's important: holds software **compiled/installed from source**, kept separate from package-manager-installed software to avoid conflicts/overwrites on updates.

```
/usr/local/apache2
/usr/local/bin
```

4. Linux File Types

Linux is famous for the philosophy "**everything is a file**" — not just documents, but devices, pipes, and sockets too. You can identify a file's type from the **first character** in `ls -l` output.

```
ls -l
-rw-r--r--  1 user user  120 Aug 11 file.txt
drwxr-xr-x  2 user user 4096 Aug 11 folder/
lrwxrwxrwx  1 user user   7 Aug 11 link -> target
brw-rw----  1 root disk   8,  0 Aug 11 sda
crw-rw-rw-  1 root root   1,  3 Aug 11 null
prw-r--r--  1 user user   0 Aug 11 mypipe
srwxr-xr-x  1 user user   0 Aug 11 mysocket
```

Symbol (1st char in <code>ls -l</code>)	File Type	Description	Example
-	Regular file	Normal file — text, binary, images, scripts	<code>file.txt</code> , <code>photo.jpg</code>
d	Directory	A folder that contains other files/directories	<code>/home/user/</code>
l	Symbolic link (symlink)	Shortcut/pointer to another file's path	<code>ln -s target link</code>
b	Block device	Hardware device that transfers data in blocks (chunks); supports random access	<code>/dev/sda</code> (hard disk)
c	Character device	Hardware device that transfers data as a stream of characters , one at a time	<code>/dev/tty</code> , <code>/dev/null</code>
p	FIFO (named pipe)	Allows one-way communication between two unrelated processes	Created via <code>mkfifo</code>
s	Socket	Enables communication between processes (often over a network or locally)	<code>/var/run/docker.sock</code>

Symbolic Link (l) points to another file by **path**; breaks if target is deleted/moved.

Block Device (b) represents storage hardware (hard disks, SSDs, USB drives). Data is read/written in **fixed-size blocks**, and random access (jumping to any block) is possible.

Character Device (c) represents devices that send/receive data as a continuous **stream**, one character at a time — no random access. Examples: keyboards, mice, terminals, `/dev/null`, `/dev/zero`.

FIFO / Named Pipe (p) is used for **one-way** inter-process communication (IPC) — one process writes, another reads, in **First-In-First-Out** order. Unlike an anonymous pipe (`|` in bash), a named pipe has an actual path on the filesystem so unrelated processes can use it.

Socket (s) enables **bidirectional** communication between processes — locally (Unix domain socket) or across a network. Example: Docker daemon communicates with the Docker CLI via a Unix socket.

4.1 Comparison: Block vs Character Device

Feature	Block Device	Character Device
Data transfer	In fixed-size blocks/chunks	Byte/character stream
Random access	✔ Yes (can jump to any block)	✗ No (sequential only)
Buffered by kernel	✔ Yes	Usually not
Examples	Hard disks, SSDs, USB drives (<code>/dev/sda</code>)	Keyboard, mouse, terminal, <code>/dev/null</code> , <code>/dev/zero</code>

🔴 **Exam Trap:** Command to check any file's type quickly (beyond `ls -l`):

```
file filename      # tells you the type in plain English
stat filename      # detailed metadata including type
```

Linux Commands — Study Notes

1. Essential Commands

1.1 `ls` — List directory contents

Flag	Meaning
<code>-l</code>	Long listing (permissions, owner, size, date)
<code>-a</code>	Show hidden files (starting with <code>.</code>)
<code>-h</code>	Human-readable sizes (KB/MB/GB)

Flag	Meaning
-R	Recursive listing
-t	Sort by modification time
-S	Sort by file size

```
ls -lah          # most commonly used combo
ls -lt /var/log  # newest files first
```

● **Exam Trap:** `ls -l` first column shows file type + permissions, e.g. `-rwxr-xr--`. First char: `-` = file, `d` = directory, `l` = symlink.

1.2 `cp` — Copy files/directories

Flag	Meaning
-r / -R	Recursive (needed for directories)
-p	Preserve permissions, timestamps
-i	Interactive (confirm before overwrite)
-v	Verbose
-u	Copy only if source is newer

```
cp file1.txt /backup/
cp -rp projectDir/ /backup/projectDir_copy/
```

1.3 `mv` — Move / Rename

```
mv old.txt new.txt          # rename
mv file.txt /home/user/docs/ # move
mv -i a.txt b.txt           # prompt before overwrite
```

💡 `mv` = **move**, not copy — no `-r` needed for directories (unlike `cp`).

1.4 `rm` — Remove files/directories

Flag	Meaning
-r	Recursive (delete directories)

Flag	Meaning
-f	Force (no prompt, ignore non-existent files)
-i	Interactive confirm
-v	Verbose

```
rm -rf oldproject/    # ⚠ dangerous – no undo, no recycle bin
```

🔴 **Exam Trap:** `rm -rf /` (or `rm -rf /*`) can wipe the entire filesystem.

1.5 `cat` — Concatenate & display files

```
cat file.txt          # print content
cat file1 file2 > merged.txt  # merge files
cat -n file.txt        # show line numbers
cat > newfile.txt      # create file, type content, Ctrl+D to save
```

1.6 `grep` — Search text using patterns

Flag	Meaning
-i	Case-insensitive
-r	Recursive search in directories
-n	Show line numbers
-v	Invert match (show non-matching lines)
-c	Count matching lines
-E	Extended regex (egrep)
-w	Match whole word only

```
grep -in "error" logfile.txt
grep -rn "TODO" ./src/
ps aux | grep nginx
```

1.7 `find` — Search files (live filesystem search)

```
find /home -name "*.txt"
find / -type f -size +100M
find . -mtime -7          # modified in last 7 days
find . -name "*.log" -exec rm {} \;  # find + delete
```

Option	Meaning
-name	Match by filename
-type f/d	File or directory
-size	Match by size
-mtime	Modified N days ago
-exec	Run a command on results

1.8 diff — Compare files line by line

```
diff file1.txt file2.txt
diff -u file1.txt file2.txt  # unified diff (used in patches/git)
diff -r dir1/ dir2/         # compare directories
```

- < = line from file1
- > = line from file2

1.9 wc — Word count

Flag	Meaning
-l	Line count
-w	Word count
-c	Byte count
-m	Character count

```
wc -l file.txt          # count lines
cat file.txt | wc -w     # count words via pipe
```

1.10 sort — Sort lines of text

Flag	Meaning
-n	Numeric sort
-r	Reverse order
-k	Sort by column/field
-u	Unique (remove duplicates)
-t	Set field delimiter

```
sort names.txt
sort -nr scores.txt      # numeric, descending
sort -t',' -k2 data.csv  # sort CSV by 2nd column
```

1.11 head / tail — View start/end of files

```
head -n 10 file.txt      # first 10 lines
tail -n 20 file.txt      # last 20 lines
tail -f /var/log/syslog  # live-follow (used for log monitoring)
```

 **Exam Trap:** `tail -f` is heavily used for "how do you monitor logs in real time?"

2. Archive & Compression

2.1 tar — Tape Archive (bundle files, no compression by itself)

Flag	Meaning
-c	Create archive
-x	Extract archive
-z	Use gzip compression
-v	Verbose
-f	Filename (must come last before name)
-t	List contents without extracting

```
tar -czvf backup.tar.gz /home/user/project/  # create compressed archive
tar -xzvf backup.tar.gz                      # extract compressed archive
tar -tvf backup.tar.gz                       # list contents
```

 Mnemonic: **c**reate, **e**xtract, **z**ip, **v**erbose, **f**ile.

2.2 gzip / gunzip — Single-file compression

```
gzip file.txt          # creates file.txt.gz, removes original
gunzip file.txt.gz     # decompresses, restores file.txt
zcat file.txt.gz       # view content of .gz file WITHOUT extracting
```

● **Note:** **gzip** compresses **one file at a time**. To compress multiple files, **tar** them first, then **gzip** (or use **tar -z** directly).

2.3 zip / unzip — Cross-platform archive + compression

```
zip archive.zip file1 file2      # zip specific files
zip -r archive.zip myFolder/     # zip a directory (recursive)
unzip archive.zip                # extract
unzip -l archive.zip             # list contents without extracting
```

Comparison: **tar.gz** vs **zip**

Feature	tar.gz	zip
Native OS	Linux/Unix	Windows (also cross-platform)
Preserves permissions	✓ Yes	⚠ Limited
Compression + archiving	Two steps (tar + gzip) combined via -z	Single step
Common use	Backups, source distribution	Cross-platform file sharing

3. Process Commands

3.1 ps — Snapshot of running processes

```
ps          # processes in current shell
ps aux      # ALL processes, all users, detailed (most used)
ps -ef      # alternate full-format listing
```

Key columns in **ps aux**: **USER**, **PID**, **%CPU**, **%MEM**, **STAT**, **COMMAND**.

3.2 top — Live process monitor

```
top
```

- Shows real-time CPU/memory usage, updates every few seconds.
- Press **k** inside **top** to kill a process, **q** to quit.
- **htop** = improved, color, interactive version (not built-in by default).

3.3 **kill** — Terminate a process

```
kill PID          # sends SIGTERM (15) – graceful shutdown
kill -9 PID       # sends SIGKILL (9) – force kill, immediate
kill -l           # list all available signals
killall firefox   # kill by process name
```

● **Exam Trap:** **SIGTERM (15)** = polite request (process can clean up); **SIGKILL (9)** = forced, cannot be caught/ignored.

3.4 **jobs, bg, fg** — Job Control

```
sleep 100 &      # run in background (& at end)
jobs             # list background jobs of current shell
bg %1            # resume job 1 in background
fg %1            # bring job 1 to foreground
```

- **Ctrl+Z** → suspends (pauses) a foreground job.
- **bg** → resumes suspended job **in background**.
- **fg** → brings background job **to foreground**.

3.5 **nohup** — Run command immune to hangups

```
nohup python3 script.py &
```

- Keeps a process running even after you **log out** or close the terminal.
- Output redirected to **nohup.out** by default if not specified.

● **Real-world use:** Running long server scripts/deployments over SSH so they survive disconnection.

4. Hard Link vs Soft (Symbolic) Link

Core Concept: Inodes

Every file on Linux has an **inode** — a data structure storing metadata (permissions, owner, size, pointers to data blocks) but **NOT the filename**. The filename is just a pointer stored in the directory entry, mapped to an inode number.

```
Directory Entry          Inode Table          Data Blocks
file.txt ---> inode 1234 ---> [permissions, size] ---> [actual data on disk]
```

4.1 Hard Link

```
ln original.txt hardlink.txt
```

- Creates a **new directory entry pointing to the SAME inode** as the original.
- Both filenames are equal — there's no "original" vs "copy"; deleting one leaves the data accessible via the other.
- File data is deleted only when **link count reaches 0** (all hard links removed).

4.2 Soft Link (Symbolic Link)

```
ln -s original.txt softlink.txt
```

- Creates a **new inode** that simply stores the **path** to the target file (like a shortcut).
- If the original file is deleted or moved, the symlink becomes a **"dangling"/broken link**.

Comparison

Feature	Hard Link	Soft Link (Symlink)
Command	<code>ln target link</code>	<code>ln -s target link</code>
Inode	Shares SAME inode as target	Has its OWN new inode
Points to	Data blocks directly	Path/filename of target
Works across filesystems/partitions	✗ No	✓ Yes
Can link to a directory	✗ No (usually restricted)	✓ Yes
If original deleted	Data still accessible (link count > 0)	Symlink breaks ("dangling link")
<code>ls -l</code> indicator	Normal file, link count shown	Shown as <code>l</code> , with <code>-></code> pointing to target

● **Exam Trap:** "Why can't hard links cross filesystems?" → Because inode numbers are only unique **within a single filesystem/partition**. A hard link is literally a reference to an inode number, which has no meaning on a different filesystem. Soft links work across filesystems because they just store a **text path**, not an inode reference.

Check link count via `ls -l` — the number after permissions (e.g., `-rw-r--r-- 2 user user ...`) shows how many hard links point to that inode.

5. `man`, `whatis`, `whereis`, `locate`, `find`

5.1 `man` — Manual pages (full documentation)

```
man ls
man 5 passwd      # section 5 = file formats
```

5.2 `whatis` — One-line description of a command

```
whatis ls
# ls (1) - list directory contents
```

5.3 `whereis` — Locate binary, source, and man page

```
whereis ls
# ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

5.4 `locate` — Fast search using a pre-built index/database

```
locate httpd.conf
sudo updatedb      # manually refresh the index database (mlocate.db)
```

5.5 `find` — Live, real-time filesystem search

```
find / -name "httpd.conf" 2>/dev/null
```

Comparison: `locate` vs `find`

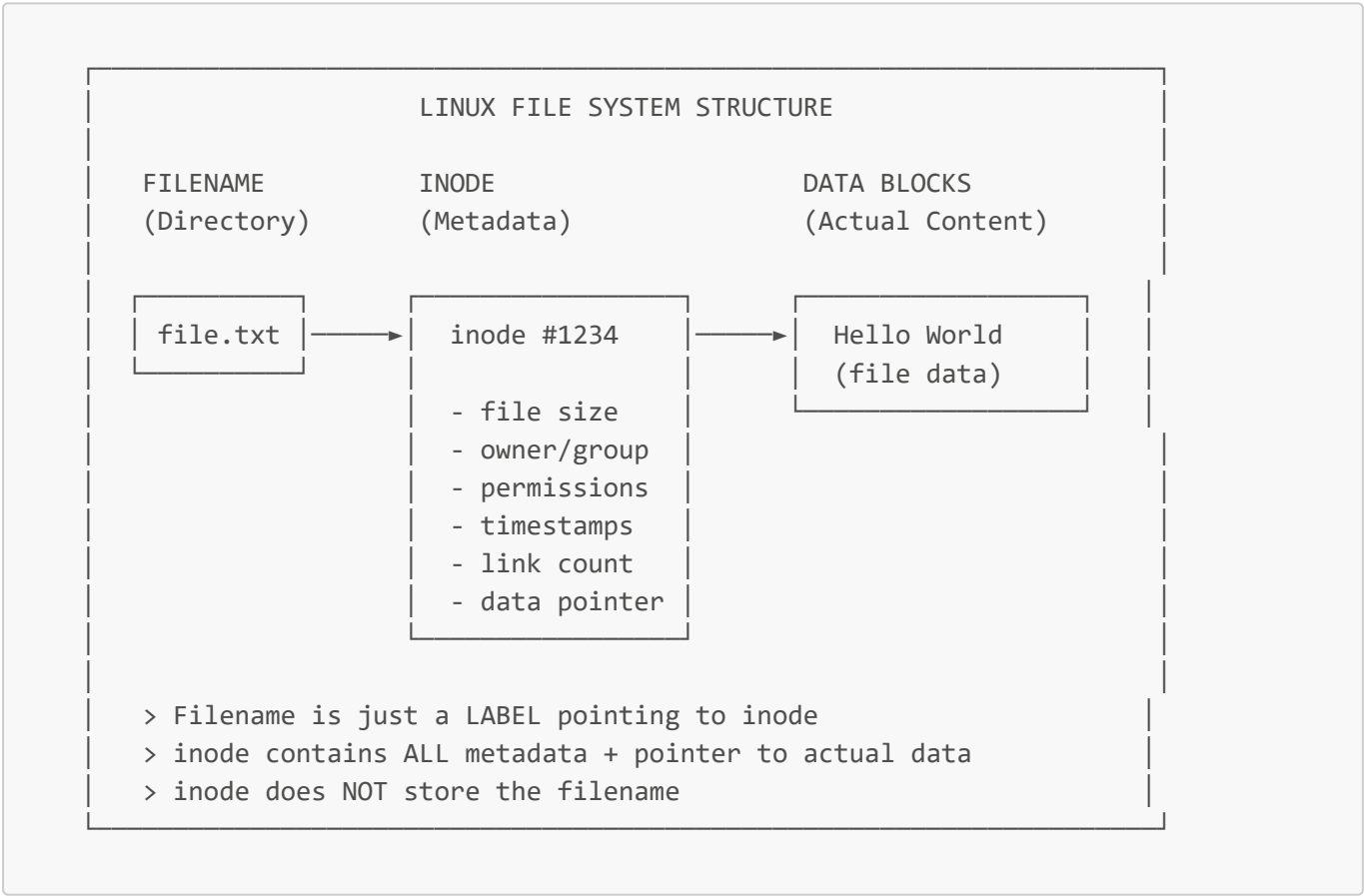
Feature	<code>locate</code>	<code>find</code>
Search method	Searches a pre-built index/database (<code>mlocate.db</code>)	Searches the live filesystem in real time
Speed	 Very fast	 Slower (scans disk directly)
Accuracy	May be outdated if <code>updatedb</code> hasn't run recently	Always up-to-date/accurate

Feature	locate	find
New/recently created files	❌ May NOT show up until index updates	✅ Always shows up
Search criteria	Filename only	Name, size, type, time, permissions, owner, and more (very flexible)
Needs root/sudo to update DB	sudo updatedb	Not needed

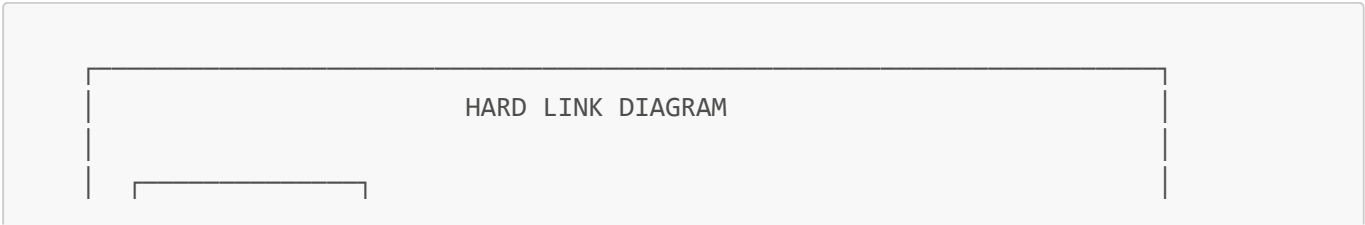
🔴 **Exam Trap:** "Why did *Locate* not find a file I just created?" → Because the *mlocate.db* index hasn't been refreshed yet. Run *sudo updatedb*, or use *find* instead, which always searches live.

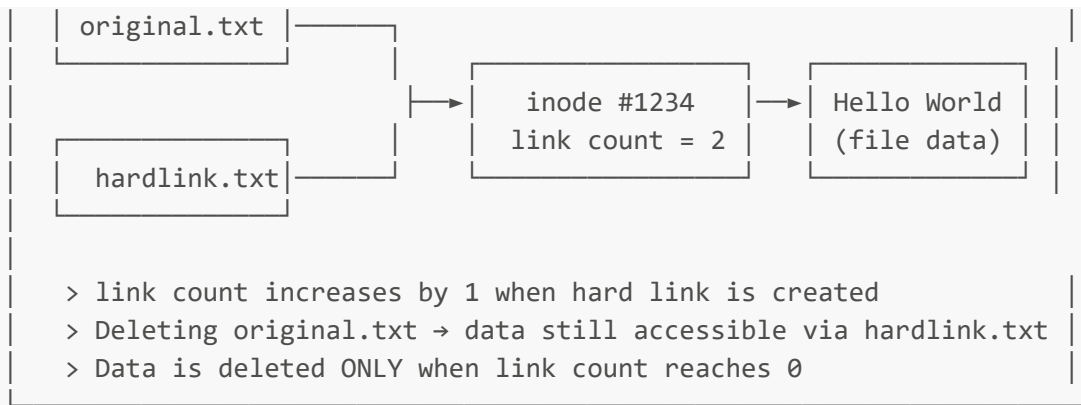
Linux File Links — Hard Link vs Soft Link (Deep Dive)

1. How Linux Stores Files (Foundation Concept)



2. Hard Link — Visualized





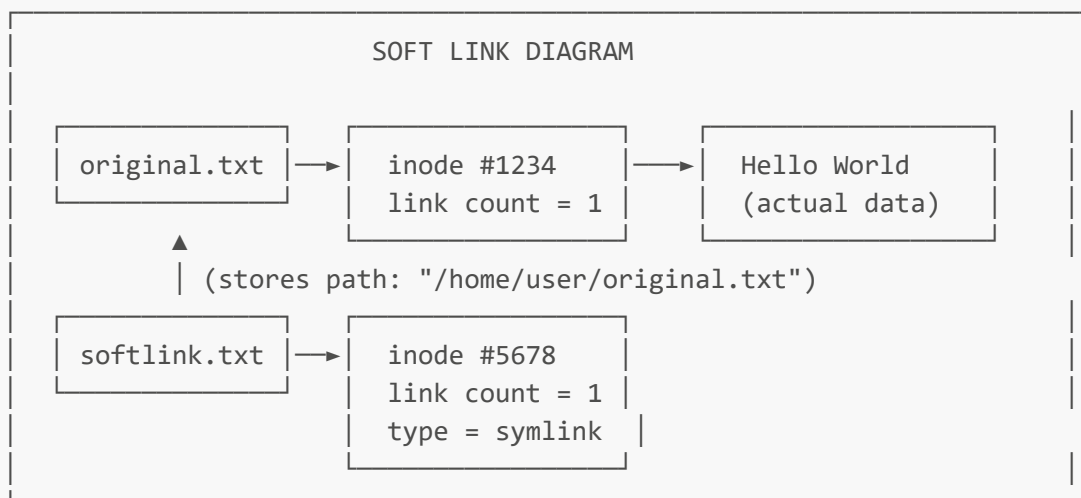
What happens when original is deleted?

BEFORE DELETE:
original.txt ———→
hardlink.txt ———→ inode #1234 (link count=2) → "Hello World"

AFTER DELETE original.txt:
[original.txt removed]
hardlink.txt —————→ inode #1234 (link count=1) → "Hello World"

- > Data is STILL SAFE – accessible via hardlink.txt
- > inode is deleted only when link count = 0

3. Soft Link (Symbolic Link) — Visualized



What happens when original is deleted?

```
BEFORE DELETE:
original.txt → inode #1234 → "Hello World"
      ▲
      | (path stored)
softlink.txt → inode #5678

AFTER DELETE original.txt:
[original.txt removed] → inode #1234 DELETED → data GONE

      ▲ (path stored – but target is gone!)
      |
softlink.txt → inode #5678 ← DANGLING LINK ⚠

> softlink.txt now points to a NON-EXISTENT file
> Accessing softlink.txt gives: "No such file or directory"
> This is called a DANGLING / BROKEN symlink
```

4. Commands & Verification

Create Hard Link

```
ln original.txt hardlink.txt
```

Create Soft Link

```
ln -s original.txt softlink.txt
# -s = symbolic (soft)
```

View inode numbers (to verify)

```
ls -li
# -l = long format
# -i = show inode number
```

Output example:

```
$ ls -li

1234 -rw-r--r-- 2 user user 12 Jun 24 original.txt
1234 -rw-r--r-- 2 user user 12 Jun 24 hardlink.txt
5678 lrwxrwxrwx 1 user user 12 Jun 24 softlink.txt -> original.txt
```

```
└─ symlink shows -> target
└─ link count (2 = hard link exists)
└─ file type: l = symlink, - = regular file
└─ inode number (1234 same for both hard links)
```

Linux File Permissions & ACLs — Study Notes

1. Permission Notation: Symbolic vs Octal

Every file/directory has **3 permission sets**: Owner (u), Group (g), Others (o) — each can have Read (r), Write (w), Execute (x).

1.1 Symbolic Notation

```
-  rwx  r-x  r--
1   2    3    4
```

1 = File type (- file, d directory, l symlink)
2 = Owner permissions
3 = Group permissions
4 = Others permissions

Symbol	Meaning (file)	Meaning (directory)
r	Read file contents	List directory contents (ls)
w	Modify/delete file contents	Create/delete files inside
x	Execute the file (run as program/script)	Enter the directory (cd)
-	Permission denied	Permission denied

1.2 Octal (Numeric) Notation

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1
None (-)	0

```
rwX = 4+2+1 = 7
rw- = 4+2   = 6
```

```
r-x = 4+1 = 5
r-- = 4 = 4
```

Common Octal Combos

Octal	Symbolic	Meaning
755	<code>rxr-xr-x</code>	Owner: full, Group/Others: read+execute (typical for scripts/binaries)
644	<code>rw-r--r--</code>	Owner: read+write, Group/Others: read-only (typical for regular files)
700	<code>rx-----</code>	Owner only — private script/dir
600	<code>rw-----</code>	Owner only, no execute — private files (e.g., SSH keys)
777	<code>rxrxrxrx</code>	Everyone full access — ● dangerous, avoid on production
666	<code>rw-rw-rw-</code>	Everyone read+write, no execute — also risky

● **Exam Trap:** 755 for directories, 644 for regular files = safe Linux defaults. 777 anywhere = major red flag in a security audit.

💡 **Quick memory trick:** r=4, w=2, x=1 → just like binary bit positions (100, 010, 001).

2. `chmod` — Change Mode (Permissions)

2.1 Numeric (Octal) Mode

```
chmod 755 script.sh      # rxr-xr-x
chmod 644 file.txt       # rw-r--r--
chmod -R 755 /var/www/   # recursive - apply to all files/subdirs
```

2.2 Symbolic Mode

Operator	Meaning
+	Add permission
-	Remove permission
=	Set exact permission

```
chmod u+x script.sh      # add execute for owner (u)
chmod g-w file.txt       # remove write for group (g)
chmod o=r file.txt       # set others to read-only
chmod a+x script.sh      # add execute for all (a = u+g+o)
chmod ug+rw file.txt     # add read+write for owner AND group
```

Target	Meaning
u	User/owner
g	Group
o	Others
a	All (u+g+o)

● **Note:** Symbolic mode is useful when you want to change **one** permission without affecting others. Numeric mode always resets **all three** sets at once.

3. `chown` — Change Owner

```
chown alice file.txt           # change owner to alice
chown alice:developers file.txt # change owner AND group together
chown -R alice:developers /data/ # recursive – whole directory tree
chown :developers file.txt      # change group only (alt to chgrp)
```

Syntax: `chown <user>:<group> <file>`

4. `chgrp` — Change Group

```
chgrp developers file.txt
chgrp -R developers /project/ # recursive
```

Comparison

Command	Changes	Needs Root/Sudo?
<code>chmod</code>	Permission bits (rwx)	Only owner or root
<code>chown</code>	Owner (and optionally group)	Root only (normal users can't give away files)
<code>chgrp</code>	Group only	Owner (if member of target group) or root

● **Exam Trap:** A regular user **cannot** `chown` a file to another user — only root can transfer ownership.

5. `umask` — Default Permission Mask

`umask` decides the **default permissions** given to newly created files and directories — it works by **subtracting** from the maximum possible permission.

Maximum defaults (before `umask` is applied):

- Files: **666** (rw-rw-rw-) — files never get execute by default
- Directories: **777** (rwxrwxrwx)

Formula:

```
Final permission = Maximum default - umask
```

```
umask          # view current umask value
umask 022      # set umask (common default)
```

Example: **umask 022**

Type	Max default	Umask	Result
File	666	022	644 (rw-r--r--)
Directory	777	022	755 (rwxr-xr-x)

 **How to calculate quickly:** umask digit tells you what to **remove** from each permission set (7-0=7, 7-2=5, 6-2=4, 6-0=6, etc.) — subtract per-digit, not a simple whole-number subtraction.

 **Exam Trap:** **umask 022** is the standard Linux default → gives owner full access, group/others read-only (and no write) on new files. **umask 077** = private by default (owner only) — often used for sensitive user home directories.

6. ACL — Access Control Lists

Why ACL?

Traditional permissions (**rwx** for owner/group/others) only allow **ONE** owner and **ONE** group per file.

Problem: What if you need to give a *specific* extra user (not the owner, not in the group) read access — without changing the file's actual group or making it world-readable?

Solution: ACL — lets you assign permissions to **multiple specific users/groups** on the same file.

6.1 **setfacl** — Set ACL permissions

```
setfacl -m u:bob:rwx file.txt      # give user 'bob' rwx access
setfacl -m g:interns:r-- file.txt  # give group 'interns' read-only
setfacl -x u:bob file.txt          # remove bob's ACL entry
setfacl -b file.txt                # remove ALL ACL entries (reset)
setfacl -R -m u:bob:rwx /project/  # recursive
```

Flag	Meaning
-m	Modify (add/update) an ACL entry
-x	Remove a specific ACL entry
-b	Remove all ACL entries
-R	Recursive
-d	Set default ACL (inherited by new files in a directory)

6.2 `getfacl` — View ACL permissions

```
getfacl file.txt
```

Example output:

```
# file: file.txt
# owner: alice
# group: developers
user::rw-
user:bob:rw-      <- extra ACL entry for bob
group::r--
mask::rw-
other::r--
```

Comparison: Traditional Permissions vs ACL

Feature	Traditional (<code>chmod</code>)	ACL (<code>setfacl</code>)
Users supported	1 owner only	Multiple specific users
Groups supported	1 group only	Multiple specific groups
Granularity	Coarse (owner/group/others)	Fine-grained (per user/group)
Command	<code>chmod</code> , <code>chown</code>	<code>setfacl</code> , <code>getfacl</code>
Viewing	<code>ls -l</code>	<code>getfacl</code> (also <code>ls -l</code> shows a <code>+</code> after permissions if ACL is set)

 **Tip:** If `ls -l` shows `rwxr-xr-c-+` — that trailing `+` means the file **has an ACL** applied beyond normal permissions.

6.3 ACL `mask` — Effective Permission Limiter

The **mask** entry defines the **maximum effective permissions** that any *named user* or *named group* ACL entry can have — it acts like a ceiling/cap.

Important: Even if you grant **rwX** to a user via ACL, if the **mask** is only **r-x**, the user's **effective** permission is capped at **r-x** (write is blocked).

```
setfacl -m u:bob:rwX file.txt      # bob granted rwX
setfacl -m mask::r-x file.txt      # mask caps everyone at r-x
getfacl file.txt
# user:bob:rwX    #effective:r-x    <- capped by mask!
```

● **Exam Trap:** "You gave a user **rwX** via ACL, but they still can't write. Why?" → Because the **mask** entry is restricting the effective permission. Always check **getfacl** output for **#effective:** — if it differs from the granted permission, the **mask** is the cause.

- **setfacl -m mask::rwX file.txt** → widen the mask to allow full effective permissions.
- The mask does **NOT** affect the file **owner** or **others** — only named users/groups and the owning group.

7. World-Writable Files — Security Risk

A **world-writable** file/directory means the "Others" (**o**) permission set includes write (**w**) — i.e., permission like **666**, **777**, **-rw-rw-rw-**, or **drwxrwxrwx**.

Why it's dangerous:

- **Any** user on the system (not just owner/group) can modify or delete the file.
- Attackers with low-privilege access can **inject malicious code** into scripts, configs, or binaries that get executed later (often by root via cron or services) → **privilege escalation**.
- World-writable directories let attackers **plant files** (e.g., fake libraries, malicious scripts).

Finding world-writable files:

```
find / -perm -o+w -type f 2>/dev/null      # world-writable FILES
find / -perm -o+w -type d 2>/dev/null      # world-writable DIRECTORIES
find / -perm -002 -type f 2>/dev/null      # alternate syntax (numeric)
```

Flag	Meaning
-perm -o+w	Match files where "others" has write permission
-type f	Only regular files
-type d	Only directories
2>/dev/null	Suppress "permission denied" errors while scanning

Fixing it:

```
chmod o-w file.txt      # remove write access for others
```

● **Exam Trap:** A **world-writable cron script** owned by root is a classic privilege escalation path — any user can edit the script, insert malicious commands, and wait for root's cron job to execute it with root privileges.

● **Special case:** `/tmp` is world-writable **by design** (needed for all users to create temp files), but it's protected using the **Sticky Bit** (`chmod +t /tmp`) — this ensures only the file's **owner** (or root) can delete/rename it, even though everyone can write into the directory.

Special Permission Bits in Linux — SUID, SGID, Sticky Bit

1. Overview

Apart from the normal `rwX` permissions (read, write, execute) for owner/group/others, Linux has **three special permission bits** that control extra behavior:

Bit	Numeric Value	Applies To
SUID (Set User ID)	4	Executable files
SGID (Set Group ID)	2	Executable files & Directories
Sticky Bit	1	Directories

These are added as a **4th digit** in front of normal permissions. Example: `chmod 4755 file` → `4` = SUID, `755` = normal `rwX` permissions.

2. SUID — Set User ID (4)

What it means

When a file with SUID is executed, the process runs with the **permissions of the file's owner**, not the user who launched it.

Why it's used

Allows a normal user to perform a privileged task (usually requiring root) through a controlled program, **without giving the user permanent root access**.

Command

```
chmod u+s filename
```

```
chmod 4755 filename
```

ls -l symbol

- Owner's **x** becomes **s** → execute + SUID present
- Owner's **x** becomes **S** (capital) → SUID set but **no execute permission** underneath (broken/unusual state)

```
-rwsr-xr-x 1 root root 64152 May 30 2024 /usr/bin/passwd
```

Real example (from `/etc/shadow` case)

- `/etc/shadow` → `rw-r-----` `root shadow` → normal users have **no access**
- `/usr/bin/passwd` → has SUID → runs as **root** temporarily
- Result: a normal user can change their own password without ever having direct access to `/etc/shadow`

Analogy

Like a bank attendant with vault access — you hand over your request, the attendant (SUID program) goes in with **their own permission**, does the task, and comes back. You never get vault access yourself.

Risk

- **High security risk if misused.**
- Any SUID root-owned script/binary is a potential **privilege escalation** point.
- If an SUID program has a vulnerability (buffer overflow, shell escape, etc.), an attacker can gain root shell.
- Common VAPT/audit check: `find / -perm -4000 -type f 2>/dev/null` → lists all SUID files on the system.

3. SGID — Set Group ID (2)

What it means

- On an **executable file**: process runs with the permissions of the **file's group**.
- On a **directory**: any new file/folder created inside automatically inherits the **directory's group** (instead of the creator's own primary group).

Why it's used

Mainly for **shared team/project directories** — ensures all files created by different users automatically belong to the same group, so the whole team can access them without manual `chgrp` every time.

Command

```
chmod g+s dirname  
chmod 2775 dirname
```

ls -l symbol

- Group's **x** becomes **s** → execute + SGID present
- Group's **x** becomes **S** (capital) → SGID set but no group execute permission

```
drwxrwsr-x 2 root devteam 4096 Jan 1 12:00 team_project
```

Real example

/data/team_project shared by multiple developers:

```
sudo chmod 2775 /data/team_project
```

Now any file created inside (by any user) automatically belongs to group **devteam**, so the whole team can read/write it — no permission mismatch.

Risk

- **Moderate risk.**
- SGID on an executable (like SUID) can allow privilege escalation if the group has elevated access.
- SGID on directories is generally safe and is a common, recommended practice for collaborative folders.
- Audit check: `find / -perm -2000 -type f 2>/dev/null` → lists all SGID files.

4. Sticky Bit (1)

What it means

When set on a **directory**, only the **file's owner** (or root) can delete or rename files inside — even if other users have write permission on the directory itself.

Why it's used

Protects files in a **publicly writable shared directory** from being deleted or renamed by other users.

Command

```
chmod +t dirname  
chmod 1777 dirname
```

ls -l symbol

- Others' x becomes t → execute + sticky bit present
- Others' x becomes T (capital) → sticky bit set but no execute permission for others

```
drwxrwxrwt 10 root root 4096 Aug 10 09:00 /tmp
```

Real example (tested)

Shared folder /shared_dir used by veena and siddhant:

Without sticky bit (777):

```
# siddhant can delete veena's file
rm /shared_dir/veena_file.txt # succeeds - BAD
```

With sticky bit (1777):

```
sudo chmod 1777 /shared_dir
rm /shared_dir/veena_file.txt
# Output: rm: cannot remove 'veena_file.txt': Operation not permitted
```

Classic real-world example: /tmp — world-writable, but sticky bit stops users from deleting each other's temp files.

Risk

- Low security risk** — it only restricts deletion, doesn't grant any extra privilege.
- Misconfiguration (forgetting to set it on a public directory) can lead to users tampering with/deleting others' files — mostly an availability/integrity issue, not a privilege escalation issue.

5. Combined Comparison Table

Feature	SUID	SGID	Sticky Bit
Applies to	Executable files	Files & Directories	Directories
Effect	Process runs as file owner	Process runs as file group / new files inherit dir's group	Only owner can delete/rename their files
Numeric value	4	2	1
Symbol in ls -l	s / S in owner's execute spot	s / S in group's execute spot	t / T in others' execute spot

Feature	SUID	SGID	Sticky Bit
Set command	<code>chmod u+s file</code>	<code>chmod g+s dir</code>	<code>chmod +t dir</code>
Numeric example	<code>chmod 4755 file</code>	<code>chmod 2775 dir</code>	<code>chmod 1777 dir</code>
Typical real use	<code>passwd</code> , <code>ping</code> , <code>sudo</code>	Shared team/project folders	<code>/tmp</code> , public upload folders
Security risk level	High (privilege escalation)	Moderate (mainly on executables)	Low (only affects delete rights)
Audit command	<code>find / -perm -4000 -type f</code>	<code>find / -perm -2000 -type f</code>	<code>find / -perm -1000 -type d</code>

Key notes: all three bits occupy the **4th (leftmost) digit** in `chmod XYZW` notation (X = special bit). Lowercase (`s`, `t`) = special bit + underlying execute permission both present. Uppercase (`S`, `T`) = special bit set but **execute permission missing**. SUID/SGID are about **identity change during execution**; sticky bit is about **restricting deletion**, not execution.

Linux User & Group Management — Study Notes

1. User Management Commands

1.1 `useradd` / `adduser` — Create a user

```

useradd veenay                                # basic user creation (no home dir on some
distros)
useradd -m veenay                             # create with home directory
useradd -m -s /bin/bash -c "Veenayak" veenay  # full setup: shell + comment
adduser veenay                                # Debian/Ubuntu – interactive, friendlier
wrapper

```

Flag	Meaning
<code>-m</code>	Create home directory
<code>-s</code>	Set login shell (e.g., <code>/bin/bash</code>)
<code>-c</code>	Comment/full name
<code>-d</code>	Custom home directory path
<code>-g</code>	Primary group
<code>-G</code>	Supplementary groups (comma-separated)
<code>-u</code>	Specify custom UID

Comparison: `useradd` vs `adduser`

Feature	<code>useradd</code>	<code>adduser</code>
Type	Low-level binary (all Linux)	High-level Perl script (Debian/Ubuntu only)
Interaction	Non-interactive, needs flags	Interactive — prompts for password, name, etc.
Home dir created by default	 No (needs <code>-m</code>)	 Yes, automatically

 **Exam Trap:** `useradd` alone does NOT create a home directory or set a password — many beginners forget `-m`.

1.2 `usermod` — Modify an existing user

```
usermod -aG developers veenay    # ADD to supplementary group (append)
usermod -s /bin/zsh veenay       # change login shell
usermod -l newname oldname       # rename user (login name)
usermod -d /new/home -m veenay   # change home dir & move contents
usermod -L veenay                # lock account (like passwd -l)
usermod -U veenay                # unlock account
```

 **Exam Trap:** `usermod -G developers user` **without** `-a` will **REPLACE** all existing supplementary groups with just `developers` — always use `-aG` (append) unless you intentionally want to overwrite group membership.

1.3 `userdel` — Delete a user

```
userdel veenay                  # delete user, KEEP home directory
userdel -r veenay               # delete user AND home directory + mail spool
```

1.4 `groupadd` / `addgroup` — Create a group

```
groupadd developers
addgroup developers             # Debian/Ubuntu equivalent
groupadd -g 1500 developers     # custom GID
```

1.5 `groupmod` — Modify a group

```
groupmod -n newname oldname    # rename group
groupmod -g 2000 developers     # change GID
```

1.6 `groupdel` — Delete a group

```
groupdel developers
```

● **Note:** You cannot delete a group that is still set as a user's **primary** group — remove/change that first.

2. Password, Identity & Login-History Commands

2.1 `passwd` — Set/change password

```
passwd veenay          # set/change password (as root, for another user)
passwd                 # change your OWN password
passwd -l veenay       # lock account (prepends ! to hash in /etc/shadow)
passwd -u veenay       # unlock account
passwd -e veenay       # force password expiry (must change at next login)
passwd -S veenay       # show password status
```

2.2 `chage` — Manage password aging

```
chage -l veenay          # list current aging info
chage -M 90 veenay       # max 90 days before password must change
chage -m 7 veenay        # min 7 days before it CAN be changed again
chage -W 7 veenay        # warn user 7 days before expiry
chage -E 2026-12-31 veenay # set account expiry date
chage -d 0 veenay        # force password change at NEXT login
```

Flag	Meaning
------	---------

-l	List aging details
----	--------------------

-M	Max days password is valid
----	----------------------------

-m	Min days before change allowed
----	--------------------------------

-W	Warning days before expiry
----	----------------------------

-E	Account expiry date
----	---------------------

-d	Last change date (0 = force change now)
----	---

2.3 `id` — Show UID, GID, and groups

```
id veenay
# uid=1001(veenay) gid=1001(veenay) groups=1001(veenay),27(sudo),1002(developers)
```

2.4 groups — Show group memberships

```
groups veenay
# veenay : veenay sudo developers
```

2.5 who — Who is currently logged in

```
who
```

2.6 whoami — Show current effective user

```
whoami
```

💡 Useful after `su/sudo` to confirm which user context you're actually in.

2.7 last — Login history

```
last                # show recent login history (from /var/log/wtmp)
last veenay         # login history for a specific user
last -x            # include shutdown/reboot events
```

🔴 **Exam Trap:** `last` reads from `/var/log/wtmp` — a classic **forensic/log-analysis** question: "How do you check who logged in last week?" → `last`.

3. /etc/passwd — User Account Database

Format: `username:x:UID:GID:comment:home:shell`

```
veenay:x:1001:1001:Veenayak:/home/veenay:/bin/bash
```

Field	Meaning	Example
1. Username	Login name	veenay
2. Password placeholder	Always <code>x</code> — real hash lives in <code>/etc/shadow</code>	<code>x</code>

Field	Meaning	Example
3. UID	User ID (0 = root, 1–999 = system, 1000+ = normal users)	1001
4. GID	Primary Group ID	1001
5. Comment (GECOS)	Full name/description	Veenayak
6. Home directory	Path to user's home	/home/veenay
7. Shell	Login shell (or /sbin/nologin to block login)	/bin/bash

● **Exam Trap:** UID 0 = root ALWAYS, regardless of username. If any non-root-named account has UID 0, that's a major security red flag (backdoor account).

● **Note:** /sbin/nologin or /bin/false in the shell field = account can't get an interactive shell — common for **service accounts** (e.g., www-data, nginx).

4. /etc/shadow — Password & Aging Database

Format: username:hash:lastchange:min:max:warn:inactive:expire:reserved

```
veenay:$6$abc123...xyz:19850:7:90:7:::
```

Field	Meaning
1. Username	Matches /etc/passwd
2. Password hash	Encrypted password (e.g., \$6\$ = SHA-512)
3. Last changed	Days since Jan 1, 1970 (epoch) when password last changed
4. Min days	Minimum days before password can change again
5. Max days	Maximum days password is valid
6. Warn days	Days before expiry to warn user
7. Inactive days	Grace period after expiry before account disabled
8. Expire date	Absolute account expiry date (epoch days)
9. Reserved	Unused

Special Hash Field Values

Value	Meaning
\$6\$...	Normal encrypted password (SHA-512)
! (prefix)	Account locked (e.g., !\$6\$abc...)
!!	Password never set (new account, no password yet)

Value	Meaning
*	Login via password disabled entirely (often service accounts)

● **Exam Trap:** Only **root** should be able to read `/etc/shadow` (permission `640` or `600`, owned by root). If it's world-readable, an attacker can grab password hashes and crack them offline (hashcat/John the Ripper).

`/etc/passwd` vs `/etc/shadow`

Feature	<code>/etc/passwd</code>	<code>/etc/shadow</code>
Contains	Account info (UID, GID, home, shell)	Password hash + aging policy
Readable by	All users (world-readable)	Root only
Password stored?	No (just <code>x</code> placeholder)	Yes (hashed)

5. `/etc/group` — Group Database

Format: `group_name:x:GID:member_list`

```
developers:x:1002:veenay,alice,bob
```

● **Note:** A user's **primary** group (set via GID in `/etc/passwd`) does **NOT** need to be listed in the member list of `/etc/group` — only supplementary memberships appear there.

6. `su` vs `sudo`

Feature	<code>su</code>	<code>sudo</code>
Password required	Target user's password	Your own password
Switches full identity?	Yes, becomes that user	Runs single command as another user (usually root)
Logging	Minimal by default	Detailed logging in <code>/var/log/auth.log</code> or <code>secure</code>
Typical use	<code>su - root</code> to become root fully	<code>sudo apt update</code> for one-off admin commands
Config file	None (PAM-based)	<code>/etc/sudoers</code> (fine-grained control)

```
su root          # switch to root, needs ROOT's password
sudo apt update  # run one command as root, needs YOUR OWN password
```

● **Exam Trap:** "Why is **sudo** generally considered safer than **su**?" → **sudo** requires the **user's own password** (root password stays secret/unused), logs every command run, and can be restricted to specific commands via **/etc/sudoers** — **su** gives full, unrestricted root access with no command-level logging by default.

7. **su** - vs **su**

```
su veenay      # switch user, but KEEP current shell's environment
su - veenay    # switch user with a FULL LOGIN shell (fresh environment)
```

Feature	su user	su - user
Environment variables	Inherited from current shell (old \$PATH , \$HOME , etc.)	Reset to the target user's own environment
Working directory	Stays in current directory	Changes to target user's home directory
Runs login scripts (.bash_profile , etc.)	✗ No	✓ Yes

● **Exam Trap:** **su - user** is the "correct"/safe way to fully become another user (especially root) — using plain **su user** can carry over the wrong **\$PATH**, potentially running unintended binaries (PATH-based privilege confusion).

💡 **Memory trick:** the **-** = "start fresh, like a real login."

8. **/etc/sudoers** & **visudo**

8.1 Why use **visudo** instead of editing directly?

```
visudo      # ALWAYS use this to edit /etc/sudoers
```

visudo **locks the file** while editing and **checks syntax** before saving — a broken **/etc/sudoers** file (e.g., from a typo) can lock EVERYONE out of **sudo**, including root recovery via **sudo**. Never use **vim /etc/sudoers** directly.

8.2 Key Syntax Examples

```
# user    host = (runas) commands
veenay    ALL = (ALL) ALL           # veenay: full sudo, any host, any user, any
command

%wheel    ALL=(ALL) ALL             # any user in "wheel" group: full sudo

veenay    ALL=(ALL) NOPASSWD: ALL   # sudo WITHOUT password prompt (risky!)
```

```
veenay ALL=(ALL) /usr/bin/systemctl restart nginx # sudo for ONE specific
command only
```

Part	Meaning
%wheel	% prefix = this is a group , not a user
ALL (host)	Applies on any host machine
(ALL)	Can run command as any target user
ALL (command)	Any command allowed
NOPASSWD:	Skips password prompt — use very sparingly

● **Exam Trap:** **NOPASSWD** sudo access is a major security risk if the account is ever compromised — an attacker gets instant root with no extra credential needed. Restrict it to very specific commands/scripts only, never **ALL**.

● **Note:** The **wheel** group is the traditional Linux/Unix group whose members are allowed to **su/sudo** to root — common on RHEL/CentOS. Debian/Ubuntu often use the **sudo** group instead.

9. Primary vs Supplementary Groups

- **Primary group:** Every user has exactly **ONE**. Defined by the GID field in **/etc/passwd**. New files created by the user get this group by default.
- **Supplementary (secondary) groups:** A user can belong to **MANY**. Listed in **/etc/group**'s member list. Used to grant extra access (e.g., **sudo**, **developers**, **docker**).

```
id veenay
# uid=1001(veenay) gid=1001(veenay) groups=1001(veenay),27(sudo),1002(developers)
```

newgrp — Temporarily switch active primary group

```
newgrp developers # starts a new shell where "developers" is the ACTIVE primary
group
```

- Useful when a user is in multiple groups but needs new files to be created with a **different** group ownership temporarily, without permanently changing their primary group.
- Exit the shell (**exit**) to return to the original group context.

Comparison

Feature	Primary Group	Supplementary Group
Count per user	Exactly 1	0 or more
Defined in	<code>/etc/passwd</code> (GID field)	<code>/etc/group</code> (member list)
New file ownership	Uses this group by default	Not used unless via <code>newgrp</code> or explicit <code>chgrp</code>
View via	<code>id -g</code>	<code>id -G</code> or <code>groups</code>

● **Exam Trap:** `id -g` shows only the **primary** group; `id -G` (capital G) shows **all** groups (primary + supplementary).

Linux Boot Process & Systemd — Study Notes

1. Key Abbreviations

Abbreviation	Full Form
BIOS	Basic Input Output System
UEFI	Unified Extensible Firmware Interface
MBR	Master Boot Record
GPT	GUID Partition Table
GUID	Globally Unique Identifier
GRUB	Grand Unified Bootloader
FHS	Filesystem Hierarchy Standard
LVM	Logical Volume Manager
PV/VG/LV	Physical Volume / Volume Group / Logical Volume

2. Core Booting Concepts

Term	Meaning
Booting	The process from computer power-on to OS startup
Bootable device	A storage device (disk, USB, CD/DVD) whose boot block contains a bootstrap program
Bootstrap program	Loads the OS kernel into RAM and starts execution; different per OS/version; located in the first sector (512 bytes) of a disk/partition
Bootloader program	Shows multiple boot options to the user; based on selection, runs the corresponding bootstrap program
Firmware	Set of programs fixed in Base ROM (motherboard) — BIOS or UEFI

Term	Meaning
Bootstrap loader	A program from firmware that finds the bootable device (as per boot device priority in BIOS/UEFI setup) and starts its bootloader

Well-Known Bootloaders (by OS)

Bootloader	OS
<code>ntldr</code> → <code>boot.ini</code>	Windows (before Vista)
<code>bootmgr</code> → <code>bcd</code>	Windows (Vista onwards); managed via <code>bcdedit</code>
LiLo (Linux Loader)	Older Linux
GRUB → <code>menu.lst</code> / <code>grub.cfg</code>	Modern Linux
BTX (BooT eXtended)	BSD Unix
SILO (Sparc Interactive Loader)	Solaris
Bootcamp	Mac OS X
uBoot	Embedded Linux

● **Exam Trap:** Bootstrap ≠ Bootloader. **Bootstrap program** = fixed, tiny, loads the next stage; **Bootloader program** = the user-facing menu (e.g., GRUB screen) that lets you pick an OS/kernel.

3. BIOS vs UEFI

BIOS (Basic Input Output System)

- Firmware standard stored on the motherboard's ROM chip.
- Runs on power-on: tests hardware, then runs the bootloader.
- Has access to basic input device ports (keyboard, mouse).
- Dominant standard for decades; being replaced by UEFI.

Functions: Initialize hardware (CPU, RAM, disk) → Perform **POST** (Power-On Self Test) → Load bootloader from disk.

UEFI (Unified Extensible Firmware Interface)

- Modern replacement for BIOS.
- Runs **faster**, supports more memory, larger storage drives, more hardware types, and better security.
- Most modern motherboards/PCs ship with UEFI by default.

Functions: Initialize hardware → Load OS → Provide advanced features (Secure Boot, network boot, etc.)

EFI Partition: Special partition at `/boot/efi` — stores bootloader files, used exclusively by UEFI systems.

Comparison

Feature	BIOS	UEFI
Age	Older (legacy)	Modern replacement
Speed	Slower	Faster
Max disk size supported	~2TB (MBR limit)	Much larger (GPT)
Security	Basic	Secure Boot support
Partition table used	MBR	GPT
Boot storage	Boot sector (512B)	EFI partition (<code>/boot/efi</code>)
Network boot	Limited	✅ Supported

● **Exam Trap:** UEFI's **Secure Boot** feature verifies that only digitally-signed/trusted bootloaders and kernels can run — protects against bootkit malware.

4. GRUB / GRUB2 — Boot Loader Deep Dive

GRUB (**GR**and **U**nified **B**ootloader) loads in **stages**, each one small enough to be loaded by the previous (very limited) stage, progressively gaining enough capability to find and load the OS kernel:

Stage	Component	Function
Stage 1	Boot sector program (MBR)	Fixed 512-byte program loaded directly by firmware on startup. Too small to understand filesystems — its only job is to locate and load Stage 1.5 (or Stage 2 directly on some setups)
Stage 1.5	Intermediate loader	Sits in the small gap right after the MBR; contains just enough filesystem drivers to locate and load Stage 2 from <code>/boot</code>
Stage 2	Second-stage boot loader	The actual GRUB menu/environment; reads <code>grub.cfg</code> , understands filesystems, loads the selected kernel + <code>initramfs</code> into RAM, and hands off control

● **Note:** Some older or simplified references collapse this into just "boot loader installer" — the utility (`grub-install`) that writes Stage 1/1.5 into the MBR and Stage 2 into `/boot/grub`. That's an **installation-time tool**, not a fourth boot-time stage. There are **3** GRUB stages.

GRUB2 Configuration

Item	Detail
Config file location	<code>/boot/grub2/grub.cfg</code> (RHEL/CentOS) or <code>/boot/grub/grub.cfg</code> (Debian/Ubuntu)
Template/source file	<code>/etc/default/grub</code> (edit this, NOT <code>grub.cfg</code> directly)
Regenerate config	<code>grub2-mkconfig -o /boot/grub2/grub.cfg</code> (RHEL) or <code>update-grub</code> (Debian)

```
# /etc/default/grub - common settings
GRUB_TIMEOUT=5          # seconds to show boot menu before default boots
GRUB_DEFAULT=0          # which menu entry boots by default (0 = first)
GRUB_CMDLINE_LINUX="..." # kernel boot parameters
```

● **Exam Trap:** Never hand-edit `grub.cfg` directly — it's **auto-generated**. Edit `/etc/default/grub`, then regenerate with `grub2-mkconfig / update-grub`. Any manual edit to `grub.cfg` gets overwritten on the next regeneration.

GRUB Rescue Mode

- If GRUB fails to find its config or the boot files are corrupted, it drops into **GRUB rescue mode** — a minimal command-line prompt (`grub rescue>`).
- Used to manually locate the boot partition and repair GRUB.

```
grub rescue> ls                # list available partitions
grub rescue> set root=(hd0,1)  # manually set root partition
grub rescue> set prefix=(hd0,1)/boot/grub
grub rescue> insmod normal
grub rescue> normal
```

● **Note:** GRUB rescue mode is a common real-world troubleshooting scenario — e.g., after a dual-boot Windows install overwrites the MBR/GRUB.

5. Full Boot Sequence (High-Level)

```
BIOS/UEFI → MBR/GPT → GRUB2 → Kernel → initramfs → systemd → Target → Login
```

Stage	What happens
BIOS/UEFI	Firmware initializes hardware, runs POST, finds bootable device
MBR/GPT	Partition table read; boot sector/EFI partition located
GRUB2	Bootloader shows menu, loads selected kernel + initramfs into RAM
Kernel	Linux kernel starts executing, initializes CPU/memory/devices
initramfs	Temporary root filesystem in RAM; loads drivers needed to mount the REAL root filesystem
systemd	First real process (PID 1) starts; brings system to a target state
Target	Final desired state reached (e.g., multi-user, graphical)
Login	Login prompt (CLI or GUI) appears; system ready for use

6. `initramfs` — Initial RAM Filesystem

Purpose: A small, temporary root filesystem loaded into **RAM** by GRUB alongside the kernel — it exists to solve a chicken-and-egg problem.

Why does `initramfs` exist?

The kernel needs **drivers** (e.g., for disk controllers, RAID, LVM, encrypted volumes) to mount the **real** root filesystem — but those drivers themselves might live *on* that real root filesystem. `initramfs` breaks this deadlock by providing just enough of a minimal environment (with the needed kernel modules) to:

1. Detect and load necessary drivers/modules (disk controller, filesystem type, LVM, RAID, etc.)
2. Locate and mount the **actual** root filesystem
3. Switch control over to it

Key Tools

Tool	Purpose
<code>dracut</code>	Modern tool used to generate/build the <code>initramfs</code> image on RHEL/Fedora-based systems
<code>pivot_root</code>	System call/command used to switch from the temporary <code>initramfs</code> root to the real root filesystem, once it's mounted

```
dracut --force                # rebuild initramfs for the current kernel
lsinitrd /boot/initramfs-*.img # inspect contents of an initramfs image
```

● **Exam Trap:** "Why can't the kernel just mount the real root filesystem directly?" → Because the drivers needed to access the real root disk (e.g., a RAID controller driver, or LVM logical volume mapping) might not be built into the kernel itself and instead live as modules on that very disk. `initramfs` provides a minimal RAM-based environment with just enough drivers to bootstrap access to the real filesystem — solving the chicken-and-egg problem.

● **Note:** `pivot_root` is conceptually similar to `chroot`, but specifically designed for the boot-time handoff from a temporary root (`initramfs`) to the permanent one — it also properly unmounts/cleans up the old root.

7. Startup Process — Who Starts First?

```
Linux kernel → accesses root filesystem → systemd/init (PID = 1) executes
```

- **systemd** (or legacy `init`) lives in `/sbin/` and is always the **very first user-space process**, given **PID 1**.
- It starts multiple **child processes** as per the configured runlevel/target.
- Runlevels/targets act like "**checkpoints**" or "**states**" the system moves through.

`systemd` vs legacy `init` (SysV)

Feature	SysV <code>init</code>	<code>systemd</code>
Service startup	Sequential (one at a time)	Parallel (much faster boot)
Config style	Shell scripts in <code>/etc/rc.d/rcX.d/</code>	Declarative unit files
PID	1	1
Speed	Slower	Faster
Dependency handling	Manual/ordered scripts	Automatic dependency resolution

8. Runlevels vs systemd Targets — Full Mapping

Traditional **SysV init** used numbered **runlevels**; modern **systemd** uses named **targets**. Both represent a "state" the system boots into.

Runlevel	systemd Target	Description
0	<code>poweroff.target</code>	Shutdown / Halt the system
1	<code>rescue.target</code>	Single-user mode — failsafe/rescue, minimal services
2	<code>multi-user.target</code> (no exact 1:1 equivalent)	Multi-user mode, login enabled, no networking
3	<code>multi-user.target</code>	Multi-user, networking , CLI only (no GUI)
4	<i>(unused/reserved)</i>	Reserved — custom use only
5	<code>graphical.target</code>	Multi-user, networking, GUI
6	<code>reboot.target</code>	Reboot the system

● **Exam Trap:** Runlevel **4** is **reserved/unused** by convention.

● **Exam Trap:** systemd collapses runlevels 2, 3, and 4 into the **same** `multi-user.target` — systemd doesn't distinguish "networking vs no networking" the way SysV runlevels 2/3 did.

Legacy Runlevel Commands

```
runlevel      # show current runlevel
init 0        # shutdown
init 6        # reboot
init 3        # switch to runlevel 3 (GUI stops if running)
init 5        # switch to runlevel 5 (GUI starts)
startx        # manually start GUI (run this from runlevel 3)
```

Modern systemd Equivalent Commands

```
systemctl get-default           # show current default target
systemctl set-default multi-user.target  # set default boot target (like
editing inittab)
systemctl isolate graphical.target    # switch to a target NOW (like `init
5`)
systemctl isolate rescue.target       # go to rescue/single-user mode
```

Comparison: Legacy vs Modern Commands

Task	SysV (legacy)	systemd (modern)
Show current level/target	runlevel	systemctl get-default
Shutdown	init 0	systemctl poweroff
Reboot	init 6	systemctl reboot
Switch to CLI multi-user	init 3	systemctl isolate multi-user.target
Switch to GUI	init 5	systemctl isolate graphical.target
Set default boot target	Edit /etc/inittab	systemctl set-default <target>

9. Linux Booting Process — Narrative Walkthrough



9.1 Power ON

When you switch on the computer, the CPU starts executing firmware code stored on the motherboard.

9.2 BIOS / UEFI

BIOS or UEFI initializes the hardware (CPU, RAM, keyboard, disk, other devices). This hardware check is called **POST** (Power-On Self-Test).

Then BIOS/UEFI looks for a bootable device (SSD, HDD, USB, Network).

9.3 Bootloader

After finding the boot device, the system starts the **bootloader** — most commonly **GRUB** (older systems: **LILO**).

GRUB can show a menu (e.g., Ubuntu, Ubuntu Advanced Options, Windows). Its main job is to load the **Linux Kernel + initramfs** into RAM.

9.4 Linux Kernel Loads

The bootloader loads the kernel into memory and gives control to it. The kernel starts managing CPU, RAM, processes, devices, drivers, and file systems.

9.5 initramfs

A small temporary filesystem loaded into RAM, containing important drivers and tools (disk driver, filesystem driver, LVM support, RAID support) needed before the real root filesystem can be mounted.

9.6 Root Filesystem Mounts

Linux finds and mounts the root filesystem `/`, after which directories such as `/etc`, `/home`, `/usr`, `/var` become available.

9.7 systemd / init Starts

The kernel starts the first user-space process — on most modern systems, `systemd`, with `PID = 1`. Older systems used `init`.

9.8 Services Start

`systemd` starts required services — network, SSH, cron, logging, database, web server, etc. E.g., `systemctl start ssh`.

9.9 Login Screen / Shell / GUI

Finally, Linux provides a login interface (CLI or GUI). After login, the user gets a shell such as Bash, and Linux is ready to use.

10. BIOS vs UEFI Boot Path, and SysV vs systemd — Side-by-Side

Legacy BIOS

```
Power ON → BIOS → MBR → GRUB → Kernel
```

Modern UEFI

Power ON → UEFI → EFI System Partition → GRUB / EFI Bootloader → Kernel

SysV vs systemd Boot — Side-by-Side

Step	SysV init	systemd
PID 1 process	/sbin/init	/usr/lib/systemd/systemd
Config source	/etc/inittab	/etc/systemd/system/default.target
Service definitions	Shell scripts (/etc/rc.d/rcX.d/)	Unit files (.service, .mount, .socket)
Execution style	Sequential	Parallel (dependency-resolved)
Speed	Slower	Faster

11. systemctl — Core Commands

systemctl is the primary tool to **manage services (units)** under systemd.

Command	Purpose
systemctl start <service>	Start a service NOW (this session only)
systemctl stop <service>	Stop a running service
systemctl restart <service>	Stop + start again (full restart)
systemctl enable <service>	Configure service to start automatically on every boot (does NOT start it now)
systemctl disable <service>	Remove service from auto-start at boot (does NOT stop it now if running)
systemctl status <service>	Show current status — running/stopped, recent logs, PID
systemctl is-enabled <service>	Check if a service is set to auto-start at boot (enabled/disabled)
systemctl daemon-reload	Reload systemd's unit file configuration after editing/adding a .service file

```
systemctl start nginx          # start nginx now
systemctl enable nginx         # make nginx start automatically on every future
boot
systemctl status nginx         # check nginx status right now
systemctl is-enabled nginx     # is nginx set to auto-start? yes/no
systemctl daemon-reload       # after editing a .service unit file, reload
systemd's cache
```

start vs enable — The Critical Distinction

Command	Effect NOW	Effect on next REBOOT
<code>systemctl start</code>	✅ Starts immediately	❌ Does nothing for future boots
<code>systemctl enable</code>	❌ Does NOT start now	✅ Will auto-start on every future boot
<code>systemctl enable --now</code>	✅ Starts immediately	✅ AND auto-starts on future boots

🔴 **Exam Trap:** "I ran `systemctl enable nginx` but `nginx` isn't running. Why?" → `enable` only creates the **symlinks** that tell systemd to start the service **on the next boot** — it does **NOT** start the service in the current session. You must also run `systemctl start nginx`, or combine both with `systemctl enable --now nginx`.

💡 **Memory trick:** `start` = "now"; `enable` = "forever (from next boot onward)". They are **independent**.

daemon-reload — When is it needed?

- Required whenever you **create, edit, or delete** a `.service` unit file.
- Without it, systemd keeps using its **cached/old** version of the unit file — your edits won't take effect.

```
vim /etc/systemd/system/myapp.service    # edit/create a custom unit file
systemctl daemon-reload                  # tell systemd to re-read unit files
systemctl start myapp                     # now start it with the updated config
```

🔴 **Exam Trap:** Forgetting `daemon-reload` after editing a unit file is one of the most common real-world systemd mistakes — the service will start with the **OLD** config until you reload.

12. Live Troubleshooting Basics

12.1 journalctl — systemd's Centralized Log Viewer

Reads logs from the **systemd journal** — a binary, structured log store (replaces plain-text `/var/log/messages` on many systems).

```
journalctl                                # show ALL logs (oldest first)
journalctl -u sshd                         # logs for a SPECIFIC service/unit
journalctl -u sshd -f                     # follow/tail logs live (like tail -f)
journalctl --since "10 min ago"           # logs from a relative time
journalctl --since today                   # logs from today only
journalctl -p err                          # only show priority "error" and above
journalctl -b                             # logs since the LAST boot only
journalctl -b -1                          # logs from the PREVIOUS boot
journalctl -k                             # kernel messages only (like dmesg)
journalctl --disk-usage                    # how much disk space journal logs are using
```

Flag	Meaning
-u	Filter by unit/service name
-f	Follow (live tail)
--since / --until	Time range filtering
-p	Filter by priority (emerg, alert, crit, err, warning, notice, info, debug)
-b	Filter by boot session
-k	Kernel messages only

🔴 **Exam Trap:** `journalctl -u sshd -f` is the go-to command to **live-debug why a service failed to start**.

12.2 `top` — Classic Live Process Monitor

Shows real-time CPU/memory usage per process, updates every few seconds by default. Inside `top`: press `k` to kill a process, `q` to quit, `M` to sort by memory, `P` to sort by CPU.

12.3 `htop` — Improved Interactive Process Monitor

Color-coded, scrollable, mouse-supported version of `top`. Not installed by default on most distros — needs `yum install htop` / `apt install htop`. Allows easily killing processes, filtering, and tree-view of parent/child processes.

Comparison: `top` vs `htop`

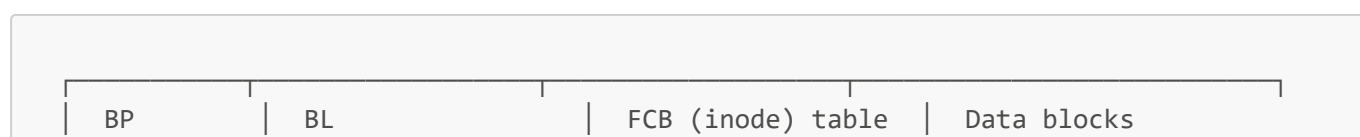
Feature	<code>top</code>	<code>htop</code>
Pre-installed	✅ Yes (almost always)	❌ No (needs install)
Interface	Plain text, keyboard-only	Color, mouse-supported, scrollable
Process tree view	❌ Limited	✅ Yes
Ease of killing processes	Press <code>k</code> , type PID	Select with arrow keys, press <code>F9</code>

💡 **Quick troubleshooting combo:** `systemctl status <service>` (is it running?) → `journalctl -u <service> -f` (why did it fail?) → `top/htop` (is something hogging CPU/RAM?).

Filesystem Partition Layout — Study Notes

1. What is a Partition Made Of?

Every disk **partition** is internally divided into **4 logical regions**, laid out in order:





Region	Generic Name	Linux-Specific Name
BP	Boot Sector	Boot block
BL	Volume Control Block	Super block
—	Master File Table (MFT)	inode list
—	Data blocks	Data blocks

💡 **Naming note:** The generic OS-textbook terms (Boot Sector, Volume Control Block, Master File Table) map directly to the Linux/Unix filesystem terms (Boot block, Super block, inode list) — same concepts, different names depending on which OS/textbook you're reading.

2. The 4 Regions Explained

2.1 Boot Sector / Boot Block

The **first region** of the partition. Contains the bootstrap code needed to start loading the OS, **if** this partition is bootable. Fixed, small size (traditionally 512 bytes for the very first sector of a disk/partition).

2.2 Volume Control Block / Super Block

Stores **metadata about the entire filesystem/partition itself** — not about individual files. Contains info like total number of blocks, free/used blocks, block size, filesystem type, pointer to the free block list, inode count (total & free).

💡 **Analogy:** If the partition were a library, the **super block** is the library's own administrative record — "how many shelves total, how many books total, how many are checked out" — NOT information about any specific book.

2.3 Master File Table (MFT) / inode List

A **table/list of FCBs (File Control Blocks)** — one entry per file on the partition. On Linux/Unix, each entry is called an **inode**; on Windows NTFS, the equivalent structure is literally called the **Master File Table (MFT)**.

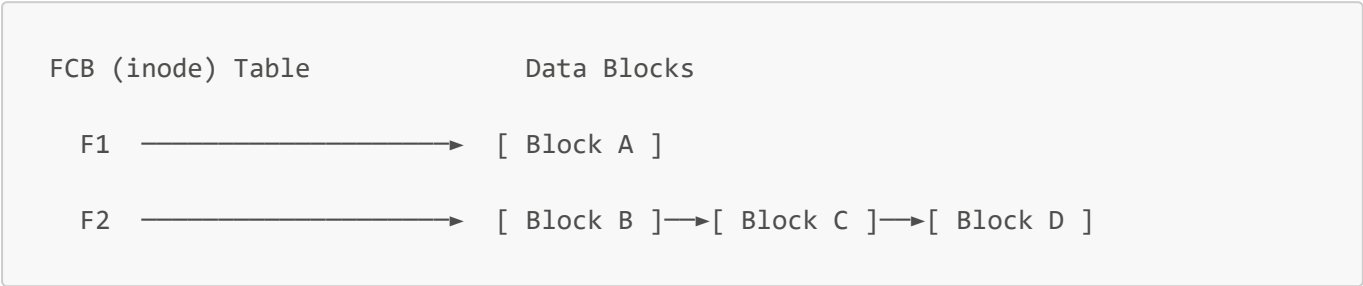
Each **FCB/inode** stores metadata about ONE file: file size, permissions (rwx), owner (UID) & group (GID), timestamps (created, modified, accessed), and **pointers to the actual data blocks** where the file's content lives.

🔴 **Exam Trap:** The inode does **NOT** store the filename! Filenames are stored separately in the **directory entry**, which just maps a name → inode number. This is exactly why **hard links** work — multiple filenames (directory entries) can point to the SAME inode.

2.4 Data Blocks

The actual **content/data** of files is physically stored here. Each FCB/inode holds **pointers** to the specific data blocks that make up that file. A single file may be spread across **multiple, non-contiguous data blocks** — the inode's pointers keep track of all of them in order.

3. FCB (File Control Block) / inode — How It Connects to Data



- **F1** → points to **1 data block** → this file's content fits in a single block.
- **F2** → points to a **chain of 3 data blocks** → this file is larger, so its content is spread across multiple blocks, linked together.

💡 **Key takeaway:** The inode/FCB is like an **index card** — it doesn't hold the file's content itself, it just holds **pointers** telling the filesystem exactly where to go find that content among the data blocks.

4. Comparison — Generic vs Linux Terms

Generic (OS Textbook) Term	Linux/Unix Term	Stores
Boot Sector (BP)	Boot block	Bootstrap code to start OS
Volume Control Block (BL)	Super block	Metadata about the WHOLE partition/filesystem
Master File Table (MFT)	inode list	One FCB/inode per file — metadata + pointers to data
Data blocks	Data blocks	Actual file content
File Control Block (FCB)	inode	Per-file metadata (size, permissions, owner, timestamps, data block pointers) — NOT the filename

DNS (Domain Name System) — Exam-Ready Notes

CDAC DITISS — Networking / Linux OS & Security

1. What is DNS?

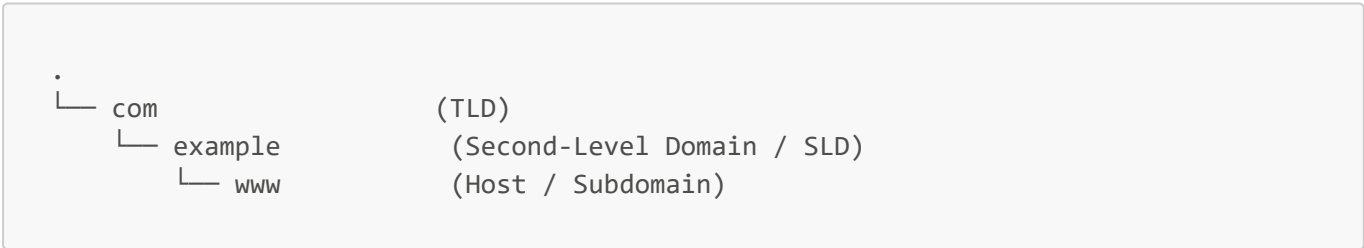
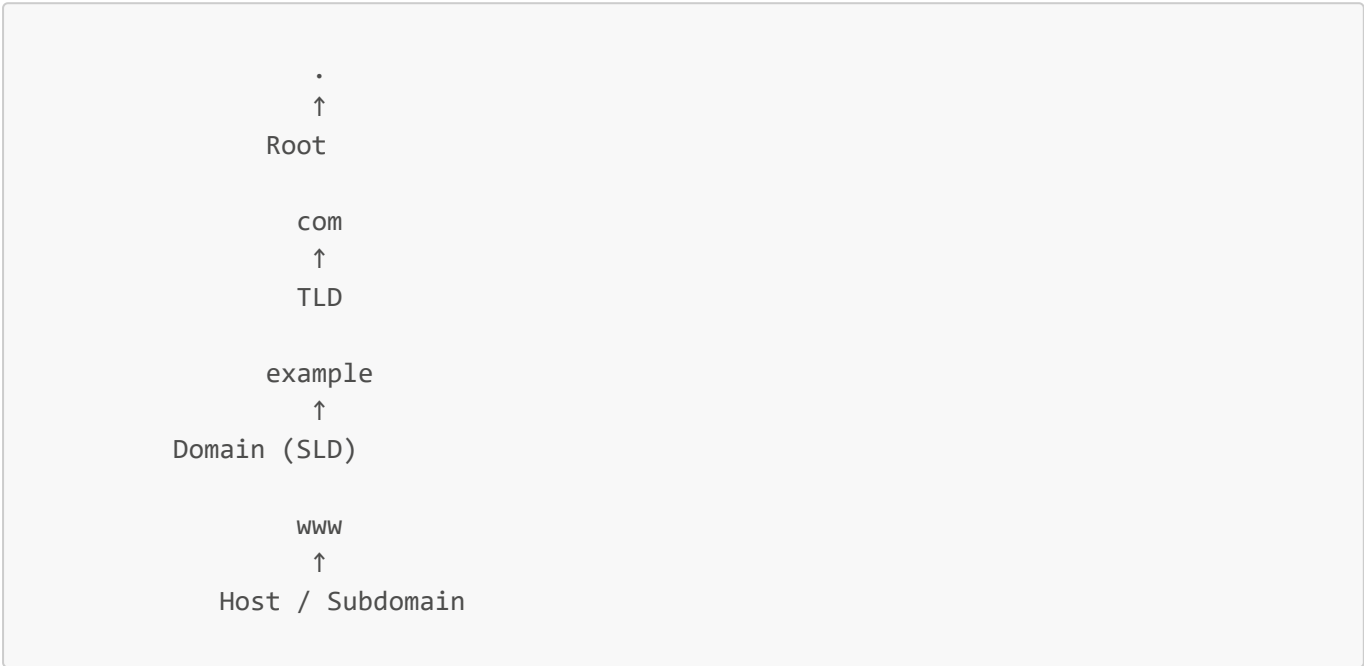
DNS translates **human-friendly domain names** into **IP addresses** that computers use to communicate.

www.example.com —DNS→ 93.184.216.34

Without DNS, you'd have to remember IP addresses instead of names.

2. DNS Naming Hierarchy

DNS is structured like an **inverted tree**, read right-to-left.



Level	Name	Example
Root Domain	The dot <code>.</code> at the very end (usually invisible)	<code>example.com.</code>
TLD (Top-Level Domain)	Highest level after root	<code>.com</code> , <code>.org</code> , <code>.net</code> , <code>.uk</code> , <code>.edu</code>
SLD (Second-Level Domain)	The registrable/main part	<code>google</code> in <code>google.com</code>
Subdomain	Optional prefix under the domain	<code>mail</code> in <code>mail.google.com</code>

Exam trap: The root is technically represented by a trailing dot (`example.com.`) — it's usually hidden by browsers/apps, but it IS part of the formal DNS hierarchy and sits **above** the TLD.

Full breakdown example — `www.example.com`:

Part	Role
. (implicit)	Root
com	TLD
example	Domain
www	Host

3. Types of TLD

Type	Full Form	Examples	Notes
ccTLD	Country Code TLD	.in (India), .uk (UK), .us (USA), .jp (Japan)	Represents a specific country
gTLD	Generic TLD	.com, .org, .net, .info	Not tied to any country
sTLD	Sponsored TLD	Managed for specific communities/sectors	Restricted registration purpose
Infrastructure TLD	—	.arpa	Used for internet infrastructure, e.g. reverse DNS

4. Key DNS Terms

4.1 Subdomain

A domain created **under** another domain.

mail.example.com

↑ ↑

subdomain domain

student.sunbeaminfo.com → subdomain under sunbeaminfo.com.

Correction note: "Anything between the host and TLD" is sometimes mis-typed as "TLS" in raw notes — it means **TLD** (Top-Level Domain), not TLS (Transport Layer Security). Don't confuse the two in exams.

4.2 Zone

A **DNS zone** is a portion of the DNS namespace managed by a particular organization/DNS server.

sunbeaminfo.com (zone)

|

├── www

└── mail



4.2a Forward Zone vs Reverse Zone

DNS zones come in two directions, depending on which way the lookup goes.

Forward Zone — maps **Name** → **IP** (the normal, everyday lookup).

```
www.example.com → 192.168.1.10
```

- Contains records like **A**, **AAAA**, **CNAME**, **MX**, **NS**, **TXT**
- Used when a browser/app looks up a website or service name

Reverse Zone — maps **IP** → **Name** (the opposite direction).

```
192.168.1.10 → www.example.com
```

- Contains **PTR** records
- Uses the special **.arpa** infrastructure domain
- IPv4 reverse zones are named using the **reversed IP octets** + **.in-addr.arpa**

Reverse zone naming example (IPv4):

```
IP address:      192.168.1.10
Octets reversed: 1.168.192
Reverse zone name: 1.168.192.in-addr.arpa
PTR record inside: 10 → www.example.com
```

Feature	Forward Zone	Reverse Zone
Direction	Name → IP	IP → Name
Main record type	A / AAAA	PTR
Special domain used	Normal domain (example.com)	in-addr.arpa (IPv4) / ip6.arpa (IPv6)
Typical use	Website/service resolution	Verifying sender identity (mail servers), logging, troubleshooting
Example zone	example.com	1.168.192.in-addr.arpa

Exam trap: A domain can have a working forward zone (**A** record) but **no** reverse zone configured (missing **PTR**) — this is common in practice and often causes mail servers to flag the sender as suspicious, since many mail servers perform a reverse-DNS check before accepting mail.

Practical/interview point — reverse lookup commands:

```
dig -x 192.168.1.10
nslookup 192.168.1.10
```

4.3 Zone File

A file containing the actual DNS records for a zone.

```
www    → 192.168.1.10
mail   → 192.168.1.20
ftp    → 192.168.1.30
```

Zone file = the DNS "database" holding name→record mappings.

4.4 Name Server

A server running DNS software. It can store records, answer queries, cache data, and refer clients to other servers.

4.5 Authoritative Name Server

The **official source** of DNS records for a domain/zone — it answers from its own data, not by asking someone else.

```
Resolver → Authoritative Server → "www.example.com = 192.168.1.10"
```

4.6 Host

A machine/service name inside a domain, e.g. `www`, `mail`, `ftp`, `server1`, `db`.

4.7 FQDN (Fully Qualified Domain Name)

The **complete** DNS name of a host.

```
www.example.com
  ↑      ↑      ↑
host  domain TLD
```

4.8 Registrar

A company through which a domain name is registered/reserved (e.g. registering `mycompany.com`). It communicates with the appropriate domain registry.

4.9 Resolver

The **client-side component** that performs DNS lookups on behalf of an application.

```
Application → DNS Resolver → DNS Servers → IP Address
```

5. Resource Records (RR)

RR = Resource Record — one individual DNS entry.

```
www.example.com A 192.168.1.10
```

RRset

Multiple records with the **same name AND same type** grouped together.

```
example.com A 10.0.0.1
example.com A 10.0.0.2
example.com A 10.0.0.3
└──────────┘
    A-record RRset
```

DNS can return all of them — useful for simple load distribution.

Parts of a Resource Record

Field	Meaning	Example
NAME	The DNS name the record belongs to	<code>www.example.com</code>
TYPE	Kind of record	<code>A, AAAA, MX, CNAME</code>
CLASS	Almost always <code>IN</code> (Internet)	<code>IN</code>
TTL	Time To Live — how long caches may keep it	<code>3600</code> (= 1 hour)
RDATA	The actual data/value of the record	<code>192.168.1.10</code>
RDLENGTH	Length of the RDATA field (protocol-level detail)	—

TTL caching flow:

```
Record received → Cached for TTL seconds → TTL expires → Query DNS again
```

6. Types of DNS Records

Record	Full Form	Purpose	Example
A	Address	Domain → IPv4	www.example.com A 192.168.1.10
AAAA	—	Domain → IPv6	www.example.com AAAA 2001:db8::10
CNAME	Canonical Name	Alias → real hostname	www.example.com → example.com
MX	Mail Exchange	Which mail server handles domain's email	example.com MX 10 mail1.example.com
TXT	Text	Domain verification, email security (SPF/DKIM etc.)	example.com TXT "verification=abc123"
NS	Name Server	Which DNS servers are authoritative for the domain	example.com NS ns1.example.com
PTR	Pointer	Reverse DNS: IP → Domain	192.168.1.10 PTR www.example.com
SOA	Start of Authority	Admin info for the zone (primary server, serial, refresh/retry/expiry)	—

Record Memory Trick

```
A      → IPv4
AAAA   → IPv6
CNAME  → Alias
MX     → Mail server
TXT    → Text/verification
NS     → Name server
PTR    → IP → Name (reverse)
SOA    → Zone administration info
```

Exam trap — MX priority: Lower numeric value = **higher preference**.

```
10 mail1.example.com ← tried first
20 mail2.example.com ← backup
```

Exam trap — PTR vs A: A record = name→IP (forward lookup). PTR record = IP→name (**reverse** lookup). Reverse DNS commonly uses the **.arpa** infrastructure TLD.

7. Types of DNS Servers (4 Key Roles)

Server	Role
Recursive Resolver	Works on behalf of the client; does all the searching and caches results
Root Name Server	Top of hierarchy; doesn't know the final IP — knows where the TLD servers are
TLD Name Server	Knows the authoritative server for a given domain
Authoritative Name Server	Holds the actual records ; gives the final answer

```
Client
  ↓
Recursive Resolver  (searches)
  ↓
Root Name Server    (knows TLD)
  ↓
TLD Name Server      (knows authoritative server)
  ↓
Authoritative Name Server (knows final record/IP)
  ↓
IP Address
```

One-line memory:

```
Resolver    → Searches
Root         → Knows TLD
TLD          → Knows authoritative server
Authoritative → Knows final record/IP
```

8. Complete DNS Resolution — Step by Step

User types `example.com` into a browser.

Step	What Happens
1. User enters domain	Browser needs the IP before it can connect
2. Check caches	Browser Cache → OS Cache → Router Cache → Resolver Cache (in that order)
3. Query Root Server	If not cached: Resolver asks Root → Root replies "ask the <code>.com</code> TLD server"
4. Query TLD Server	Resolver asks <code>.com</code> TLD → TLD replies "ask <code>example.com</code> 's authoritative server"

Step	What Happens
5. Query Authoritative Server	Resolver asks the authoritative server directly for the record
6. Response	Authoritative server returns the record, e.g. A = 93.184.216.34
7. Caching	Resolver caches the answer for the record's TTL duration
8. Deliver to browser	Resolver hands the IP back to the browser, which connects to the website

Cache Check Order (Step 2 detail)

```
Browser Cache
  ↓ (miss)
OS Cache
  ↓ (miss)
Router Cache
  ↓ (miss)
ISP / Recursive Resolver Cache
  ↓ (miss)
→ Proceed to full DNS resolution (Root → TLD → Authoritative)
```

Full Resolution Diagram

```
1. example.com typed
  ↓
2. Check caches (browser/OS/router/resolver) → not found
  ↓
3. Resolver → Root Server
   "com is here"
  ↓
4. Resolver → .com TLD Server
   "example.com's authoritative server is here"
  ↓
5. Resolver → Authoritative Server
   "What is example.com's IP?"
  ↓
6. Authoritative Server
   A = 93.184.216.34
  ↓
7. Resolver caches answer (per TTL)
  ↓
8. Resolver → Browser → 93.184.216.34 → Website loads
```

Important exam point: The **Root server does NOT return the final IP**. It only refers the resolver to the correct TLD server. Similarly, the **TLD server does NOT return the final IP** either — it refers to the authoritative server. Only the **authoritative server** gives the actual record.

Real-World Analogy

```
You → Ask information desk → "Which city?"  
    → Ask city office → "Which street?"  
    → Ask street authority → "House number is 34"
```

```
Client → Resolver → Root → TLD → Authoritative → IP
```

DHCP (Dynamic Host Configuration Protocol) — Exam-Ready Notes

CDAC DITISS — Networking / Linux OS & Security

1. What is DHCP?

DHCP = Dynamic Host Configuration Protocol — automatically gives network settings to devices, instead of manual configuration.

Manually you'd configure:

```
IP address  
Subnet mask  
Default gateway  
DNS server
```

With DHCP, this happens automatically:

```
DHCP Server (Pool: 192.168.50.100 - 192.168.50.200)  
    ↓  
Laptop connects  
    ↓  
Gets:  
IP      → 192.168.50.105  
Gateway → 192.168.50.1  
DNS     → 8.8.8.8
```

2. Why DHCP is Needed

Without DHCP, every device needs manual configuration:

```
Laptop 1 → 192.168.50.10
Laptop 2 → 192.168.50.11
Laptop 3 → 192.168.50.12
```

For hundreds of devices, this is impractical and error-prone.

With DHCP:

```
Device joins network → DHCP automatically gives configuration
```

3. DHCP Layer & Ports

- DHCP is an **Application Layer** protocol.
- Uses **UDP** — because the client may not even have an IP address yet (UDP doesn't require an established connection like TCP does).

Device	Port
DHCP Server	UDP 67
DHCP Client	UDP 68

Easy memory:

```
Server → 67
Client → 68
```

Exam trap: DHCP uses UDP, not TCP — because at the DISCOVER stage the client has no IP address, so it can't establish a TCP connection. UDP allows broadcast communication without a prior handshake.

4. Benefits of DHCP

Benefit	Explanation
Automation	New device connects → gets IP automatically, no manual work
Avoids IP conflicts	Prevents accidentally assigning the same IP to two devices
Efficient IP usage	IPs given as time-limited leases ; unused addresses get reclaimed and reused
Mobility	A laptop moving between Home → Office → College gets correct config automatically at each location
Centralized management	Gateway, DNS, domain, IP range, lease time — all configured once on the server

5. Key DHCP Components

Component	Meaning
DHCP Server	Machine that manages IP addresses (pool, gateway, DNS, lease duration, reservations)
DHCP Client	Any device requesting config — laptop, phone, desktop, printer, smart TV
Scope	The range of IPs DHCP can hand out, e.g. 192.168.50.100 – 192.168.50.200
Lease	An IP given to a client for a specific time period, e.g. 2 hours
Reservation	A specific device (identified by MAC address) always gets the same IP — a.k.a. MAC-to-IP binding
DHCP Options	Extra config sent along with the IP — gateway, DNS, subnet mask, domain name

Reservation example:

```
Printer MAC: AA:BB:CC:11:22:33
Always gets: 192.168.50.50
```

6. DHCP DORA Process (Most Important!)

DORA = Discover → Offer → Request → Acknowledge



Step 1 — DHCPDISCOVER

Client has **no IP** and **no known DHCP server**. It **broadcasts**:

```
Client:68 → broadcast → Server:67
"Is there any DHCP server available?"
```

Step 2 — DHCPOFFER

Server replies with an offer:

```
"I can give you 192.168.50.101"
```

May include: IP address, subnet mask, gateway, DNS, lease time.

Step 3 — DHCPREQUEST

Client accepts the offer:

```
"I want 192.168.50.101"
```

This is commonly **broadcast** too, so all DHCP servers on the network know which offer was accepted (and the ones not chosen can withdraw their offers).

Step 4 — DHCPACK

Server confirms:

```
"Confirmed. You can use this IP."
```

Client now has: IP, Gateway, DNS, Lease time.

Memory trick:

```
Discover → Find server  
Offer    → Server offers IP  
Request  → Client requests IP  
ACK      → Server confirms IP
```

Exam trap: DHCPREQUEST is typically **broadcast**, not unicast — this lets every DHCP server on the segment know which offer the client accepted, so the other servers can release their reserved offers back to their pools.

7. Other DHCP Messages

Message	Meaning	Example Scenario
DHCPNAK	Negative Acknowledgement — server rejects the request	Client asks for an IP that's invalid, unavailable, or the client moved to a different subnet

Message	Meaning	Example Scenario
DHCPDECLINE	Client rejects an offered IP because it's already in use	Client checks via ARP , finds the address occupied, sends DHCPDECLINE
DHCPRELEASE	Client gives back the IP when done with it	Client shuts down / disconnects → server can reuse the address
DHCPINFORM	Client already has a static IP but wants DHCP <i>options</i> (DNS, domain name, etc.) without requesting a new IP	Statically configured server that still wants DNS settings from DHCP

DHCPDECLINE flow:
 Server offers 192.168.50.110
 ↓
 Client checks using ARP
 ↓
 Address already in use!
 ↓
 Client sends DHCPDECLINE

Exam trap: Don't confuse DHCPDECLINE (client refuses a specific offered IP because it's already taken) with DHCPNAK (server refuses the client's requested IP). One is client-initiated, the other server-initiated.

8. DHCP Lease Lifecycle

Suppose lease time = **8 hours**. The client does NOT wait for the full 8 hours to try renewing — it starts early.

Stage	Timing	What Happens
1. Initialization	Boot	Client boots → runs DORA → gets IP (e.g. 192.168.50.110)
2. Normal Operation	—	Client simply uses the assigned IP normally
3. T1 (Renewal)	~ 50% of lease time (e.g. ~4 hrs of 8)	Client sends unicast DHCPREQUEST to the original DHCP server: "Can I continue using this IP?"
4. T2 (Rebinding)	~ 87.5% of lease time	If original server didn't respond, client broadcasts DHCPREQUEST — now willing to accept help from ANY DHCP server
5. Expiration	100% of lease time	If still no response, client must stop using the IP and restarts DORA from scratch

DORA → Get IP → Use IP
 ↓

```
T1 = 50% → try ORIGINAL server (unicast)
↓ (fail)
T2 = 87.5% → try ANY server (broadcast)
↓ (fail)
Lease expires → stop using IP → restart DORA
```

Exam trap: T1 renewal is **unicast to the original server**; T2 rebinding is **broadcast to any server**. This distinction (who it talks to, and how) is a classic exam/viva question.

Memory trick: T1 = "Talk to the one I know" (50%) → T2 = "Talk to anyone" (87.5%) → Expire = "Start over".

9. DHCP Server Configuration (RHEL-family)

Step 1 — Give the DHCP Server a Static IP

A DHCP server should itself have a stable, unchanging address.

```
sudo nmcli connection modify "ens160" ipv4.addresses 192.168.50.5/24
sudo nmcli connection modify "ens160" ipv4.method manual
sudo nmcli connection down "ens160"
sudo nmcli connection up "ens160"
ip address show
```

Step 2 — Install DHCP Server

```
sudo dnf update -y
sudo dnf install dhcp-server -y
```

Step 3 — Configure `/etc/dhcp/dhcpd.conf`

```
default-lease-time 600;
max-lease-time 7200;

authoritative;

subnet 192.168.50.0 netmask 255.255.255.0 {
    range 192.168.50.100 192.168.50.200;
    option routers 192.168.50.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 8.8.8.8;
}
```

Configuration Directive Breakdown

Directive	Meaning
<code>default-lease-time 600;</code>	Default lease = 600 seconds = 10 minutes
<code>max-lease-time 7200;</code>	Maximum lease = 7200 seconds = 2 hours
<code>authoritative;</code>	"I am the authoritative DHCP server for this network"
<code>subnet 192.168.50.0 netmask 255.255.255.0</code>	Defines the network <code>192.168.50.0/24</code>
<code>range 192.168.50.100 192.168.50.200;</code>	The DHCP pool — addresses that can be handed out
<code>option routers 192.168.50.1;</code>	Tells clients their default gateway
<code>option subnet-mask 255.255.255.0;</code>	Tells clients their subnet mask
<code>option domain-name-servers 8.8.8.8;</code>	Tells clients their DNS server

Resulting client config:

```
IP      → 192.168.50.100 - 200 (from pool)
Mask    → 255.255.255.0
Gateway → 192.168.50.1
DNS     → 8.8.8.8
Lease   → per configured settings
```

Step 4 — Validate Configuration

```
sudo dhcpd -t
```

Checks `/etc/dhcp/dhcpd.conf` for syntax errors **before** restarting the service.

```
Edit dhcpd.conf → dhcpd -t → No errors → Restart DHCP
```

Step 5 — Bind DHCP to an Interface

File: `/etc/sysconfig/dhcpd`

```
DHCPDARGS=ens160
```

Meaning: DHCP server should listen on interface `ens160`.

Step 6 — Start the DHCP Server

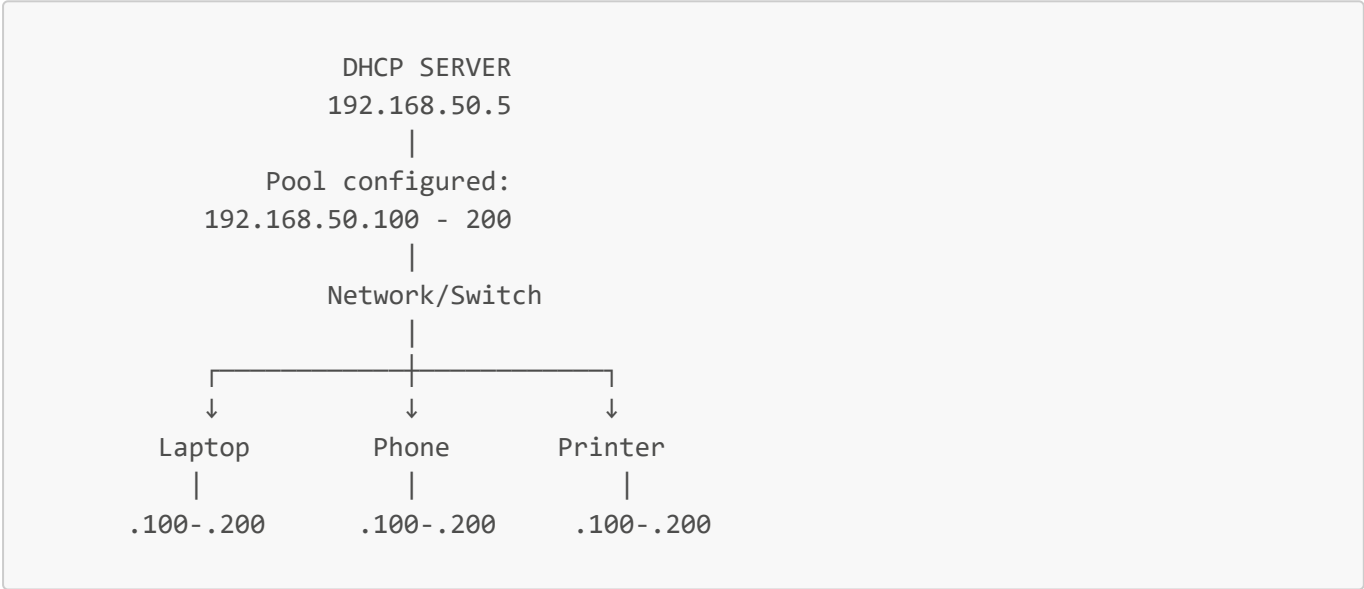
```
sudo systemctl enable --now dhcpd
systemctl status dhcpd
```

`enable` = start automatically at boot; `--now` = start it immediately too.

Step 7 — Firewall

```
sudo firewall-cmd --add-service=dhcp --permanent
sudo firewall-cmd --reload
```

10. Complete DHCP Architecture



Each device gets a unique address from the pool.

11. Best Practices

Practice	Why
DHCP server should have a static IP	If the DHCP server's own IP changes, clients lose track of where to renew leases
Don't overlap DHCP pool with statically assigned IPs	Prevents IP conflicts between manually configured devices and the DHCP pool
Avoid uncontrolled multiple DHCP servers	Two independent DHCP servers (e.g. router's built-in DHCP + a Linux DHCP server) both answering DHCPDISCOVER causes unpredictable client configuration

Example of separating static vs. pool addresses:

```
Static (outside pool):
192.168.50.1  → Router
192.168.50.5  → DHCP server
192.168.50.10 → File server
192.168.50.20 → Printer

DHCP pool:
192.168.50.100 - 192.168.50.200
```

File Sharing Protocols — FTP, NFS, Samba/SMB, TFTP

Exam-Ready Notes (CDAC DITISS — Linux OS & Security / Networking)

1. FTP (File Transfer Protocol)

1.1 Overview

FTP is used to transfer files between a client and a server. It uses **two separate connections**, not one.

Port	Purpose
------	---------

21	Control port — commands & replies (login, LIST, RETR, PORT, PASV etc.)
----	--

20	Data port (used traditionally in Active mode)
----	---

- Daemon: **vsftpd** (Very Secure FTP Daemon) — same package/service name on RHEL and Ubuntu.
- Control connection stays open for the whole session; data connection opens/closes per transfer.

```
Control port (21) → commands, login, replies
Data port      (20) → actual file/data transfer
```

Exam trap: Students often think FTP uses only port 21. Remember — port 21 is for *control*, actual data moves on a **separate** connection (port 20 in active mode, or a dynamic high port in passive mode).

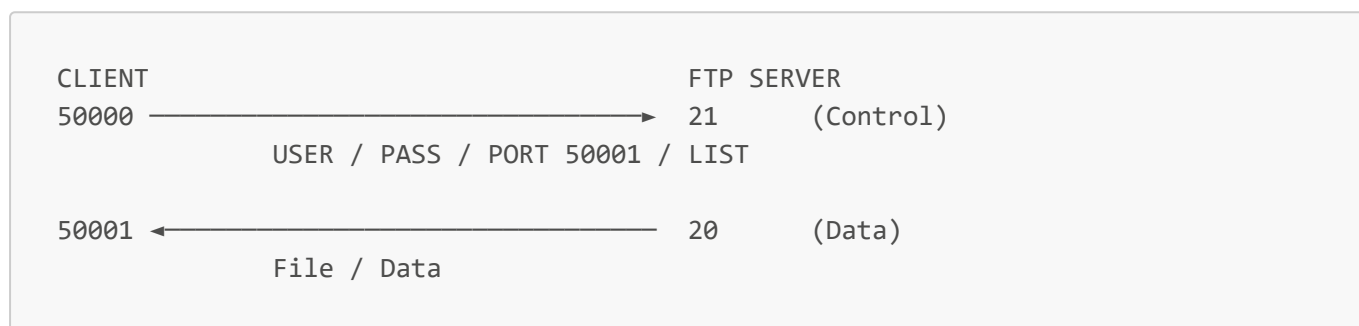
FTP has two modes of operation: **Active FTP** and **Passive FTP**. The difference is entirely about **who initiates the data connection**.

1.2 Active FTP — Full Flow

Key idea: The **SERVER** connects back to the **CLIENT** for data.

```
Client IP: 192.168.1.10      Server IP: 203.0.113.10
```

Step	Action
1	Client opens control connection: Client:50000 → Server:21
2	Client logs in: USER veenayak, PASS **** → Server replies 230 Login successful
3	Client picks a data port (e.g. 50001) and tells server using PORT command
4	Client sends a data request: LIST or RETR file.txt
5	Server initiates the data connection: Server:20 → Client:50001
6	Data (file/listing) is transferred
7	Data connection closes; control connection (21) may stay open



Problem with Active FTP: The server tries to make a **new inbound connection** to the client.

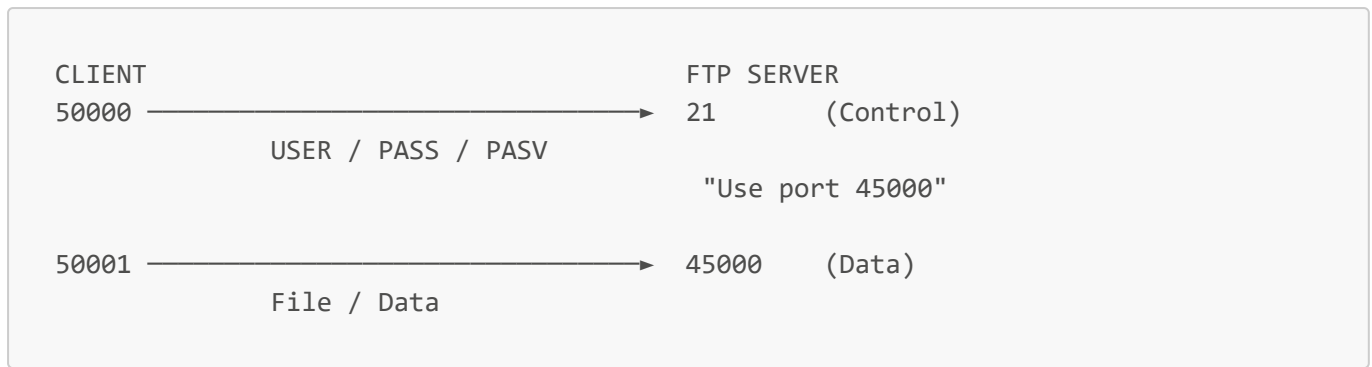
Internet → Client Firewall/NAT → Client

This inbound connection is commonly **blocked** by client-side firewalls/NAT (home routers, corporate firewalls) — because from the firewall's point of view, it's an *unsolicited incoming connection*.

1.3 Passive FTP — Full Flow

Key idea: The CLIENT initiates BOTH connections.

Step	Action
1	Client opens control connection: Client:50000 → Server:21
2	Client logs in (same as active)
3	Client sends PASV command ("give me a port to connect to")
4	Server picks a data port (e.g. 45000) and tells the client
5	Client initiates the data connection: Client:50001 → Server:45000
6	Client sends LIST / RETR file.txt
7	Data transferred over Client:50001 ↔ Server:45000
8	Data connection closes; control connection may stay open



Both connections start **from the client**, so a stateful firewall easily allows it:

Client starts connection → Allow outgoing → Allow related return traffic

Note: Passive FTP doesn't magically bypass firewalls on its own — the **server's** firewall must still open the passive port range (e.g. 50000–51000) in addition to port 21.

1.4 Active vs Passive — Comparison Table

Feature	Active FTP	Passive FTP
Control connection	Client → Server:21	Client → Server:21
Data port selected by	Client	Server
FTP command used	PORT	PASV
Data connection initiated by	Server	Client
Traditional data port	20	Dynamic high port
Firewall/NAT friendly	✗ Less	✓ More
Common today	Rare	Widely preferred

Easiest memory trick:

ACTIVE: Control → Client initiates | Data → SERVER initiates
PASSIVE: Control → Client initiates | Data → CLIENT initiates

Viva one-liner: "Active FTP: server connects back to the client for data. Passive FTP: client connects to the server for data."

2. NFS (Network File System)

2.1 What is NFS?

NFS lets a Linux system **share directories over a network** so another machine can mount and use them almost like local folders. Files physically live on the **server**; the client accesses them over the network.

```
NFS Client —Network— NFS Server
/mnt/data                /data (actual files)
```

Example:

```
sudo mount 192.168.1.10:/data /mnt/data
ls /mnt/data      # shows files physically stored on 192.168.1.10
```

2.2 NFS vs Windows File Sharing

OS Family	Protocol
Linux/Unix	NFS
Windows	SMB/CIFS

2.3 Client–Server Roles

Role	Responsibility
Server	Stores files, decides which directories are shared (exported), controls client access
Client	Requests the shared directory, mounts it locally, reads/writes per permissions

2.4 Key Terms

Export — making a server directory available to other machines. Configured in `/etc/exports`.

```
/data 192.168.1.0/24(rw, sync)
```

Meaning: share `/data` with the `192.168.1.0/24` subnet, allow read+write, write synchronously.

Mount — attaching a remote (or local) filesystem to a local directory.

```
sudo mount serverA:/home /mnt/nfs
```

Exam/practical trap: Never mount a remote share directly onto an existing important directory like `/home` — it can **hide** the client's existing local content while mounted. Use a safe empty mount point like `/mnt/nfs`.

Internal flow when reading a file over NFS:

```
Application → Linux filesystem → NFS client → Network → NFS server → Server filesystem → file.txt
```

The application doesn't handle network packets itself — the OS/NFS layer does it transparently.

2.5 RPC (Remote Procedure Call)

RPC lets one computer request a function/service from another over the network.

```
Client: "Server, perform this operation for me." → Server: "Here's the result."
```

Older NFS versions depend on several RPC-based helper services: `rpcbind`, `mountd`, `statd`, `lockd`.

2.6 NFS Versions

Feature	NFSv2	NFSv3	NFSv4
Age	Very old	Improved v2	Modern
Transport	TCP/UDP	TCP/UDP	Primarily TCP
Main port	—	2049 (+ RPC helpers)	2049
RPC helper services needed	Yes	Yes (mountd, rpcbind, lockd, statd)	Mostly not needed
File size limits	Yes	Larger support	Large
Firewall friendliness	Poor	More complex	Easier (single port)
Kerberos/security	—	Limited	Strong integration
ACL support	—	Limited	Improved
Recommended today	✗	Older systems	✓ Preferred

```
NFSv3: Client → [rpcbind, mountd, NFS, lock, status services] → Server (complex firewall rules)
NFSv4: Client → TCP 2049 → Server (simple firewall rule)
```

Exam trap: NFSv4's biggest advantage is often mis-stated as "faster." The real exam-relevant advantage is: **consolidated onto a single well-known port (2049), reducing RPC dependency and simplifying firewall configuration**, plus built-in Kerberos/ACL support.

2.7 Server Configuration (RHEL/Rocky/Alma family)

Step	Command
1. Install	<code>sudo dnf install nfs-utils -y</code>
2. Create shared dir	<code>sudo mkdir -p /nfs/share</code>
3. Edit exports	<code>sudo vim /etc/exports</code> → add <code>/nfs/share 192.168.1.0/24(rw, sync)</code>
4. Apply exports	<code>sudo exportfs -a</code> (apply) / <code>sudo exportfs -v</code> (view active exports)
5. Start service	<code>sudo systemctl enable --now nfs-server</code>
6. Check status	<code>systemctl status nfs-server</code>

2.8 Common `/etc/exports` Options

Option	Meaning
<code>ro</code>	Read-only
<code>rw</code>	Read and write
<code>sync</code>	Write changes synchronously (safer)
<code>async</code>	Faster but risk of data loss on failure
<code>root_squash</code>	Remote root user is not treated as local root (safer, default)
<code>no_root_squash</code>	Remote root keeps root privileges on server — risky

Safe example:

```
/nfs/share 192.168.1.0/24(rw, sync, root_squash)
```

2.9 Firewall Rules

```
# NFSv4 (simple – single port)
sudo firewall-cmd --add-service=nfs --permanent
sudo firewall-cmd --reload

# NFSv3 (needs extra RPC-related services)
sudo firewall-cmd --add-service=nfs --permanent
sudo firewall-cmd --add-service=mountd --permanent
sudo firewall-cmd --add-service=rpc-bind --permanent
sudo firewall-cmd --reload
```

3. Samba / SMB / CIFS

3.1 What is Samba?

Samba is software that lets **Linux and Windows** share files, printers, and authentication using the **SMB/CIFS** protocol.

```
Windows Client → SMB/CIFS → Linux Server (Samba) → Shared Folder
```

Mainly used to let **Windows clients** access files stored on a **Linux server**.

3.2 What is SMB/CIFS?

SMB = Server Message Block — the network protocol Windows primarily uses for file sharing, printer sharing, network folders, and authentication. **CIFS** is an older related name/version of SMB.

```
Windows PC → \\192.168.1.10\share → Samba Server → /var/smb/share
```

3.3 Samba Services / Daemons

Daemon	Role
smbd	Handles file sharing, printer sharing, SMB connections (main service)
nmbd	Historically handled NetBIOS name services (older setups)

3.4 Authentication Methods in Samba

Method	Description
Local Samba users	Create a Linux user + a separate Samba password: <code>sudo useradd veenayak → sudo smbpasswd -a veenayak</code>
PAM + Domain Controller	PAM (Pluggable Authentication Modules) is Linux's auth framework; can integrate with a Windows domain for centralized credentials
Samba as AD Domain Controller	Samba can act as an Active Directory DC , so Windows PCs authenticate against a Linux server (users, passwords, groups, policies)
LDAP Backend	LDAP (Lightweight Directory Access Protocol) can store centralized users/groups/passwords. <i>Modern Samba AD DC deployments typically use Samba's own built-in directory rather than plain LDAP as a password store.</i>

3.5 NFS vs Samba

NFS	Samba
Common in Linux/Unix-to-Linux environments	Common between Windows and Linux
Uses NFS protocol	Uses SMB/CIFS protocol
Default port: 2049	Default port: TCP 445

4. TFTP (Trivial File Transfer Protocol)

4.1 Overview

Property	Detail
Transport	UDP
Default port	69
Authentication	✗ None (no username/password)
Directory browsing	✗ Not supported
Complexity	Very simple/lightweight

Client → Request file (must know exact filename) → TFTP Server
Server → Sends file in small blocks → Client

4.2 Common Uses

- Network boot / **PXE boot**
- Router/switch **configuration backup**
- **Firmware** transfer
- Small boot/config files

PXE Client → TFTP Server → bootloader / kernel file

4.3 FTP vs TFTP

Feature	FTP	TFTP
Full name	File Transfer Protocol	Trivial File Transfer Protocol
Transport	TCP	UDP
Port	21 (control) + 20/dynamic (data)	69
Login/auth	✓ Username/password	✗ None
Directory listing	✓ Supported	✗ Not supported
Feature set	Rich	Minimal
Typical use	General-purpose file transfer	Boot files, firmware, config backup

Directory listing — explained simply:

```
FTP:   Client → "Show me files"           → Server → file1.txt, file2.txt, docs/
TFTP:  Client → "Give me boot.img"        → Server → sends boot.img (must already
know the name)
```

FTP supports browsing/listing files. TFTP requires you to know the file name beforehand.

Easy memory:

```
FTP  = Full-featured file transfer
TFTP = Tiny/simple file transfer
```

5. Quick Reference — Packages, Daemons & Services

Service	RHEL/Rocky/Alma Package	Ubuntu/Debian Package	Main Daemon	Common systemd Service
SMB/Samba	samba	samba	smbd, nmbd	RHEL: <code>smb</code> , <code>nmb</code> • Ubuntu: <code>smbd</code> , <code>nmbd</code>
NFS	nfs-utils	nfs-kernel-server	nfsd	RHEL: <code>nfs-server</code> • Ubuntu: <code>nfs-kernel-server</code>
FTP	vsftpd	vsftpd	vsftpd	vsftpd
TFTP	tftp-server	tftpd-hpa	in.tftpd	often <code>tftp/tftpd-hpa</code> or socket-activated

Port Cheat Sheet

Protocol	Port(s)	Transport
FTP (control)	21	TCP
FTP (data, active)	20	TCP
FTP (data, passive)	Dynamic high port	TCP
SSH/SFTP	22	TCP
TFTP	69	UDP
NFS (v3/v4)	2049	TCP/UDP
SMB	445	TCP

Mail Services — Exam-Ready Notes

CDAC DITISS — Networking / Linux OS & Security

1. What is a Mail Service?

A **mail service** is a system used to **send, receive, store, and manage** email. It works through three layers:

```
Mail Client → Mail Server → Mail Protocols
```

Category	Examples
Mail Clients	Thunderbird, Outlook, Apple Mail, <code>mail</code> command
Mail Server Software	Postfix, Dovecot, Sendmail, Procmail

2. Main Mail Components — Quick Overview

Component	Purpose
SMTP	Send email
IMAP	Read/sync email from server
POP3	Download email
Postfix	SMTP mail server / MTA
Dovecot	IMAP/POP3 server
Maildir	Stores emails as files
Procmail	Filters/delivers emails
Sendmail	Mail transfer server
Thunderbird/Outlook/Apple Mail	Mail clients

Easy memory:

```
Postfix → SEND/TRANSFER
Dovecot → READ/ACCESS
Maildir → STORE
SMTP    → SEND
IMAP    → SYNC/READ
POP3    → DOWNLOAD
```

Overall flow to remember:

```
Client → SMTP → Postfix → Mailbox/Maildir → Dovecot → IMAP/POP3 → Client
```

3. Postfix, Dovecot, Sendmail, Procmail

Software	Role	Protocol Used
Postfix	Mail transfer server — sends/receives mail between servers	SMTP
Dovecot	Lets users read/access mail stored on the server	IMAP / POP3
Sendmail	Another mail transfer agent (older, Postfix is a modern alternative)	SMTP
Procmail	Mail filtering/delivery — sorts mail into folders based on rules	—

Postfix = SMTP mail server (MTA)
Dovecot = IMAP/POP3 server

Procmail example flow:

```
Incoming mail → Procmail → Check rules → Move to correct folder
  Boss email   → Work folder
  Newsletter   → Newsletter folder
  Spam         → Spam folder
```

4. Email Protocols — SMTP, IMAP, POP3

4.1 SMTP (Simple Mail Transfer Protocol)

Job: Sending email.

```
Mail Client → SMTP → Mail Server → SMTP → Other Mail Server
```

Port	Purpose
25	Server-to-server SMTP relay
587	Mail submission (client → server)
465	SMTP over implicit TLS

Easy memory: SMTP = **S**end **M**ail

4.2 IMAP (Internet Message Access Protocol)

Job: Read/manage mail while it mainly stays on the server.

Mail Server → Laptop, Phone, Tablet (all see same mailbox)

Reading a message on one device marks it read on all devices — this is **synchronization**.

Port	Purpose
143	IMAP
993	IMAPS (IMAP over implicit TLS)

4.3 POP3 (Post Office Protocol v3)

Job: Download email from server to client.

Mail Server → Download → Laptop

Traditionally designed around **one main device** — depending on client settings, mail may be deleted from the server after download.

Port	Purpose
110	POP3
995	POP3 over TLS (POP3S)

IMAP vs POP3

Feature	IMAP	POP3
Mail location	Mainly stays on server	Downloaded to client
Multi-device sync	✔ Yes	✗ Not designed for it
Best for	Phone + laptop + webmail	One main device
Port	143 / 993	110 / 995

SMTP vs POP3 vs IMAP (Full Comparison)

Feature	SMTP	POP3	IMAP
Purpose	Send mail	Download mail	Read/sync mail
Direction	Push	Pull	Pull/sync
Server-side role	Transfer/queue	Simple retrieval	Mailbox management
Folders	N/A	Very limited	Multiple folders

Feature	SMTP	POP3	IMAP
Message flags	N/A	Limited	Yes (Seen, Replied, Flagged, Deleted)
Partial fetch	N/A	Limited	Yes
Server-side search	N/A	No/rudimentary	Yes
Multiple devices	N/A	Less suitable	Excellent
Typical secure port	587/465	995	993

Easiest possible summary:

SMTP → SEND
 POP3 → DOWNLOAD
 IMAP → SYNC

Exam trap: SMTP does **NOT** define how mail is stored or read. It only handles sending/transferring. Storage (Maildir/mbox) and reading (IMAP/POP3) are entirely separate concerns handled by different software.

5. Mail Storage — Maildir vs mbox

Mail storage format = **how emails are saved on disk** after the server receives them.

mbox

All emails stored in **ONE big file**.

```
/var/mail/bob
↓
[Mail1][Mail2][Mail3][Mail4]
```

Because many emails share one file, **locking** is required when multiple processes access it.

Maildir

Every email stored as a **separate file**.

```
/home/bob/Maildir/
├─ new/   → New/unread messages
├─ cur/   → Already processed/seen messages
└─ tmp/   → Temporary files during delivery
```

```
Maildir/new/
├─ mail001
├─ mail002
└─ mail003
```

mbox vs Maildir — Comparison

Feature	mbox	Maildir
Storage	All emails in one file	One file per email
Locking	Required	Usually not required
NFS usage	Poorer	Better
Backup	Whole mailbox	Individual messages
Delete message	Can require rewriting mailbox	Delete one file
Corruption risk	One damaged file may affect many emails	Usually affects one message
Performance (many messages)	Can become slower	Usually easier to manage

Easy memory:

```
mbox    → ONE mailbox file
Maildir → ONE file per MAIL
```

Maildir Filename Flags

Filename example:

```
1234567890.12345_0.hostname,S=1024,W=2048:2,S
```

You don't need to memorize the full format — just the **flags** at the end:

Flag	Meaning
D	Draft
F	Flagged / Starred
P	Passed / Forwarded
R	Replied
S	Seen / Read
T	Trashed

Example: `:2,S` means the email has been **Seen/Read**.

Practical flow: Postfix receives email → stored in Maildir → Dovecot reads Maildir → Thunderbird displays email.

6. Email Message Structure

An email has **three main parts**:

```
Email
├── Headers / Envelope info
├── Body
└── Attachments
```

Important Header Fields

Field	Meaning
From	Who sent the email
To	Main recipient
Cc	Carbon Copy — visible extra recipients
Bcc	Blind Carbon Copy — hidden from other recipients
Subject	Short description of email
Date	When email was sent
Attachments	Files sent with the email

Cc vs Bcc Example

```
To : Bob
Cc : Carol
Bcc : David
```

Bob and Carol can see: `Bob`, `Carol` (each other). **Neither can see David** — David is hidden.

```
Cc → visible recipients
Bcc → hidden recipients
```

Raw Email Format (RFC 5322)

```
From: Alice <alice@gmail.com>
To: Bob <bob@company.com>
Subject: Meeting tomorrow
Date: Mon, 13 Apr 2026 10:30:00
```

```
Hi Bob,
Let's meet tomorrow.
```

```
Headers
  ↓
Blank line  ← separates headers from body
  ↓
Body
```

7. Mail System Architecture — MUA, MSA, MTA, MDA

These are the **four core components** of any email system.

Component	Full Name	Role	Examples
MUA	Mail User Agent	Application the user interacts with (write/read/reply/forward/attach)	Thunderbird, Outlook, Evolution, Mutt, mail, mailx
MSA	Mail Submission Agent	Accepts outgoing mail from the MUA; checks auth, size, spam	Postfix (port 587)
MTA	Mail Transfer Agent	Transfers mail between mail servers using SMTP	Postfix, Exim, Sendmail
MDA	Mail Delivery Agent (a.k.a. LDA — Local Delivery Agent)	Delivers received mail into the correct local mailbox	Dovecot, Procmail, Cyrus IMAP, fetchmail, getmail, fdm

Easy Memory

```
MUA → User writes/reads mail
MSA → Accepts mail from user
MTA → Transfers mail between servers
MDA → Delivers mail into mailbox
```

Simple flow sequence:

MUA → MSA → MTA → MTA → MDA → Mailbox → MUA

With real software:

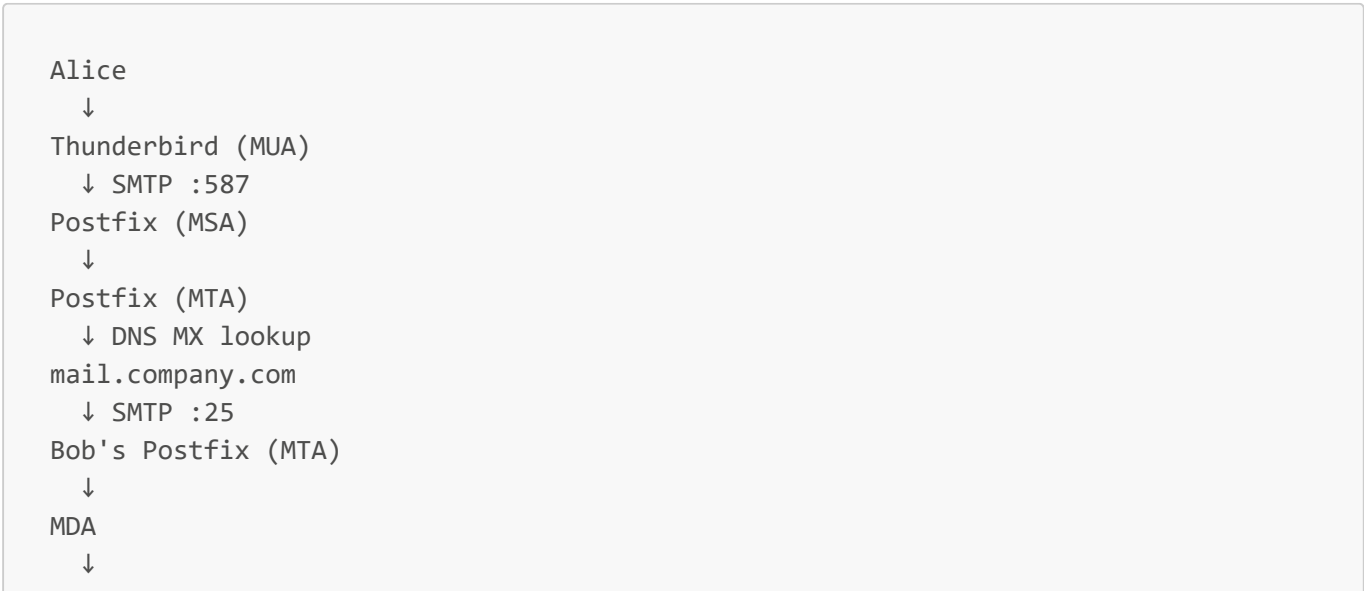
Thunderbird → Postfix → Postfix → Dovecot/mailbox → Outlook

8. Complete Email Flow — Step by Step

Scenario: `alice@gmail.com` sends mail to `bob@company.com`.

Step	What Happens	Component
1	Alice composes email in her client	MUA (Thunderbird/Gmail)
2	Client submits mail via SMTP (port 587); checks auth, size, spam	MSA
3	MSA hands off to MTA, which adds headers (<code>Received</code> , <code>Message-ID</code>)	MTA
4	MTA looks up <code>company.com</code> 's MX record via DNS to find the mail server	DNS MX lookup
5	Alice's MTA connects to Bob's MTA via SMTP (port 25) and transfers the message	Server-to-server SMTP
6	Bob's server checks: does Bob exist? spam/antivirus/filtering?	MDA
7	Message is written to disk (mbox or Maildir)	Mailbox storage
8	Bob opens his MUA, connects via IMAPS (port 993) through Dovecot, and reads the mail	MUA + Dovecot

Full Diagram



```
Mailbox (mbox / Maildir)
↓
Dovecot
↓ IMAP :993
Outlook (MUA)
↓
Bob
```

DNS MX Lookup Detail

```
bob@company.com
↓
Extract domain: company.com
↓
Query DNS for MX record
↓
company.com MX 10 mail.company.com
↓
Resolve mail.company.com → IP (A/AAAA record)
↓
20.5.6.7
```

Exam trap: MX record points to a *hostname*, not directly an IP — the resolver must then look up that hostname's A/AAAA record separately to get the actual IP.

9. Special Scenarios

Scenario A — Both Users on Same Server

If `alice@company.com` sends to `bob@company.com`, both are on the same domain. The MTA recognizes Bob is a **local user** and skips the external DNS MX lookup and internet SMTP hop entirely — goes straight to MDA → mailbox.

Scenario B — Recipient is Offline

Bob's laptop is off when Alice sends the mail. No problem — the server **stores** the email in Bob's mailbox. When Bob comes online, Outlook connects via IMAP and the email appears.

Key point: The recipient does NOT need to be online at send time.

Scenario C — Greylisting (Anti-Spam Technique)

When an unfamiliar sender first tries to deliver, the receiving server may reply:

```
450 Temporary failure – try again later
```

Legitimate mail servers retry automatically after some time; many spam systems don't retry correctly, so this filters some spam.

First attempt → 450 Try later → Few minutes later → Retry → Accepted

Scenario D — Bounce Mail (DSN)

If `bob@company.com` doesn't exist, the server responds:

550 No such user

The sending system generates a **DSN (Delivery Status Notification)**, commonly called a **bounce message**, informing Alice the delivery failed.

Alice → Send → bob@company.com ❌ → 550 No such user → Bounce/DSN → Alice

10. SMTP Commands & Responses

Example SMTP Conversation

```
Client: EHLO mail.example.com
Server: 250 OK

Client: MAIL FROM:<alice@example.com>
Server: 250 OK

Client: RCPT TO:<bob@company.com>
Server: 250 OK

Client: DATA
Server: 354 Start mail input

Client: Subject: Hello
      Hi Bob, How are you?
      .

Server: 250 Message accepted
Client: QUIT
```

SMTP Commands

Command	Purpose
HELO	Starts a basic SMTP session
EHLO	Extended/modern version of HELO — server tells client what extensions it supports (STARTTLS, AUTH, SIZE)
MAIL FROM	Specifies the sender
RCPT TO	Specifies recipient (can be repeated for multiple recipients)
DATA	Starts the actual message content; ends with a single . on its own line
RSET	Cancels current transaction without closing connection
VERFY	Attempts to verify a user (often disabled for security/privacy)
EXPN	Attempts to expand a mailing list (also commonly disabled)
NOOP	Does nothing — used to keep/check connection alive
QUIT	Ends the SMTP session

SMTP Response Codes

Code Range	Meaning	Example
2xx	Success	250 OK — command/message accepted
3xx	More info required	354 Start mail input — occurs after DATA
4xx	Temporary failure — sender should retry	450 Mailbox busy
5xx	Permanent failure — retrying won't help	550 No such user

Easy memory:

```
2xx → Success
3xx → Continue / more data
4xx → Temporary failure (retry)
5xx → Permanent failure (don't retry)
```

Exam trap: Don't confuse 4xx and 5xx — 4xx means "try again later" (transient), 5xx means "this will never work, stop retrying" (permanent). Greylisting deliberately uses 450 (4xx) to trigger legitimate retries.

11. POP3 Commands

Example POP3 Session

```
Client: USER bob
Server: +OK

Client: PASS password
Server: +OK

Client: STAT
Server: +OK 3 4500

Client: LIST
Server: message list

Client: RETR 1
Server: sends message 1

Client: DELE 1
Server: marks message for deletion

Client: QUIT
```

POP3 Commands

Command	Purpose
USER	Specifies username
PASS	Sends password
STAT	Shows mailbox summary (message count, total size)
LIST	Lists messages and sizes
RETR	Retrieves/downloads a specific message
DELE	Marks a message for deletion (actual deletion committed on successful QUIT)
RSET	Undoes deletion marks made during the current session
UIDL	Returns unique IDs for messages
QUIT	Ends the session

Exam trap: **DELE** only *marks* a message for deletion — the deletion is only finalized when the session ends successfully with **QUIT**. If the connection drops before **QUIT**, the message is NOT deleted (or **RSET** can undo the mark before quitting).

POP3 Limitations

- Mainly works around a single **INBOX** — no rich server-side folder management like IMAP
- Limited message flags (no Seen/Replied/Flagged model like IMAP)
- No rich server-side search — it's primarily a download protocol

12. SMTP Ports — Detailed

Port	Name	Encryption	Common Use
25	SMTP	STARTTLS may be used	Mail server → Mail server (relay)
587	Submission	STARTTLS commonly used	Mail client → Mail server
465	SMTPS	Implicit TLS	Secure mail submission

Easy memory:

```
25 → Server to Server
587 → Client sends mail
465 → SMTP with implicit TLS
```

13. FOSS SMTP/Mail Implementations

Protocol	Software Implementations
SMTP	Postfix, Exim, Sendmail, OpenSMTPD
IMAP/POP3	Dovecot

```
Protocol → SMTP
Software:
├─ Postfix
├─ Exim
├─ Sendmail
└─ OpenSMTPD
```

14. When to Use What

Protocol	Best When
SMTP	Sending application notifications, relaying mail between servers, sending user email, building mail transfer infrastructure
POP3	Simple mailbox download is enough; mainly one client/device used; server-side management not needed
IMAP	Using phone + laptop + webmail together; want server-side folders, flags, read/unread sync, and remote mailbox management

15. Modern Common Mail Stack (Full Picture)

```
Alice's MUA
  ↓
SMTP :587 (Submission)
  ↓
Postfix (MSA)
  ↓
SMTP :25 (Server-to-server relay, STARTTLS)
  ↓
Bob's Postfix (MTA)
  ↓
Mailbox (mbox / Maildir)
  ↓
Dovecot
  ↓
IMAP :993 (IMAPS)
  ↓
Bob's MUA
```

NIS & LDAP — Directory Services — Exam-Ready Notes

CDAC DITISS — Networking / Linux OS & Security

PART A — NIS (Network Information Service)

1. What is NIS?

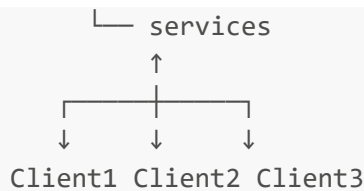
NIS = Network Information Service — an older **client-server directory service** used mainly in Unix/Linux networks to keep common system information centralized instead of maintaining it separately on every machine.

Without NIS:

```
Client1 → own users/groups
Client2 → own users/groups
Client3 → own users/groups
```

With NIS:

```
NIS Server
├─ users
├─ groups
└─ hosts
```



NIS provides centralized user and system information to multiple Unix/Linux clients.

What can NIS manage?

Central distribution of files like:

```
/etc/passwd    → user accounts
/etc/shadow    → passwords
/etc/group     → groups
/etc/hosts     → hostnames
/etc/services  → network services
/etc/protocols → network protocols
```

Instead of creating user **john** separately on 20 machines, his info is managed centrally.

2. Why is NIS also called YP?

NIS was originally called **Yellow Pages (YP)** — that's why many NIS commands/services still start with **yp**:

```
ypbind
ypserv
ypcat
ypmatch
```

NIS ≈ YP (old Unix terminology)

3. How NIS Works — Step by Step

Scenario: user **john** tries to log in.

Step	What Happens
1	Client needs info about john
2	ypbind (client-side binding service) connects the client to an NIS server
3	NIS server receives the query

Step	What Happens
4	Server searches a NIS map (e.g. <code>passwd.byname</code>)
5	Server returns John's info: <code>john:x:1005:1005:John:/home/john:/bin/bash</code>

```
Client
  ↓
ypbind
  ↓
NIS Server
```

ypbind = NIS client-side binding service

Complete NIS Flow

```
User tries: john
  ↓
Linux Client
  ↓
ypbind
  ↓
NIS Server
  ↓
Search map: passwd.byname
  ↓
Find john
  ↓
Return user information
  ↓
Client uses it
```

4. What is a NIS Map?

A **map** = a NIS database containing a particular type of information (like a table).

Map	Purpose
<code>passwd.byname</code>	User info searchable by username
<code>passwd.byuid</code>	User info searchable by UID
<code>group.byname</code>	Group info by name
<code>hosts.byname</code>	Host info by name
<code>hosts.byaddr</code>	Host info by IP address

```
NIS Map = database/table
```

5. Security Limitations of NIS (Important!)

NIS was designed for **trusted internal Unix networks** — this is its biggest weakness.

Limitation	Detail
No encryption	Data can be sniffed on the network — no built-in encrypted transport
Weak authentication	Relies on trusted hosts/network config rather than strong user/server authentication
Weak password policy support	No rich complexity/history/expiration policy features
Vulnerable to spoofing	Clients can't strongly verify they're talking to the genuine NIS server
Predictable/discoverable services	Relies on RPC, making services easy to discover on the local network
Limited access control	Designed to broadcast info broadly — fine-grained control is weak

Exam trap: NIS's core weakness isn't a single flaw — it's a **combination of design-era assumptions** (trusted internal network) that don't hold up in modern, hostile network environments.

6. NIS vs LDAP (Quick Preview)

NIS	LDAP
Older	More modern
Mainly Unix-focused	Platform independent
Weak security	TLS/security integration
Simple data maps (flat)	Hierarchical directory
Limited scalability	Better scalability

```
NIS → Old centralized directory (flat)
LDAP → Modern flexible directory protocol (hierarchical)
```

PART B — LDAP (Lightweight Directory Access Protocol)

8. What is LDAP?

LDAP = Lightweight Directory Access Protocol — a protocol to **store, search, and manage directory information** over a network.

Think of it as a **central company phonebook/database**:

```
LDAP Server
├── Users
├── Groups
├── Computers
├── Printers
└── Departments
```

Applications query this central directory instead of maintaining separate user lists.

Why "Lightweight"?

LDAP came from the older, complex **X.500 DAP (Directory Access Protocol)**, which depended on the OSI networking stack. LDAP was designed to be simpler and work directly over **TCP/IP**.

```
DAP → Complex (OSI stack)
LDAP → Lightweight/simpler (TCP/IP)
```

LDAP is a Protocol, Not Software

```
LDAP → Protocol
OpenLDAP, Active Directory, 389 Directory Server → Software implementing it
```

Same relationship as:

```
HTTP → protocol   |   Apache → software
LDAP → protocol   |   OpenLDAP → software
```

9. LDAP Ports

Port	Purpose
389	LDAP
636	LDAPS (LDAP over TLS)
3268	Microsoft AD Global Catalog

Easy memory:

389 → LDAP
636 → Secure LDAP

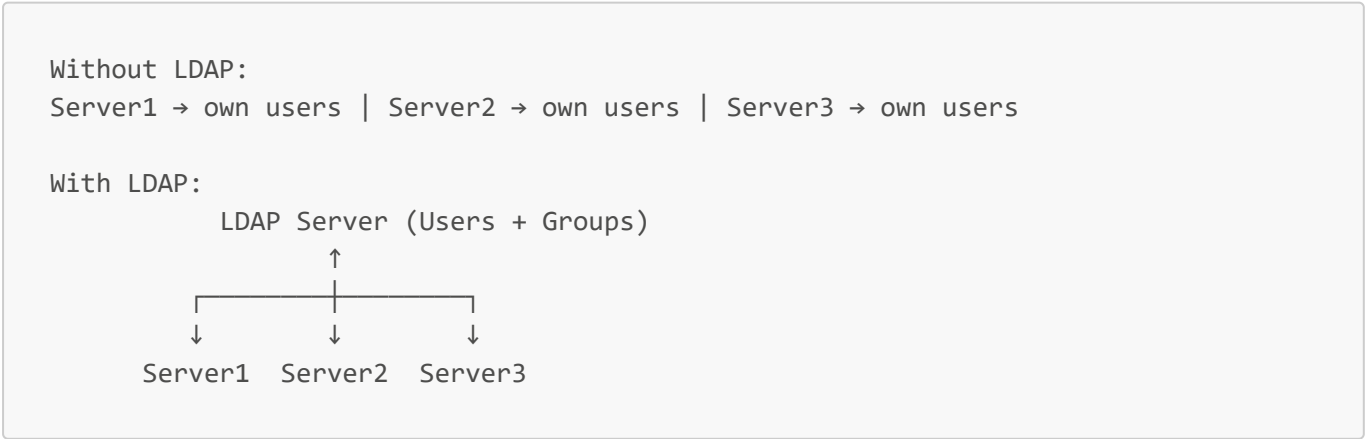
10. Common LDAP Implementations

Implementation	Notes
OpenLDAP	Open-source; main daemon = slapd
Microsoft Active Directory	Combines LDAP + Kerberos + DNS; widely used for enterprise identity
FreeIPA	Common in Linux/Red Hat environments; integrates LDAP + Kerberos + DNS + Certificate services

LDAP protocol → OpenLDAP software → slapd daemon

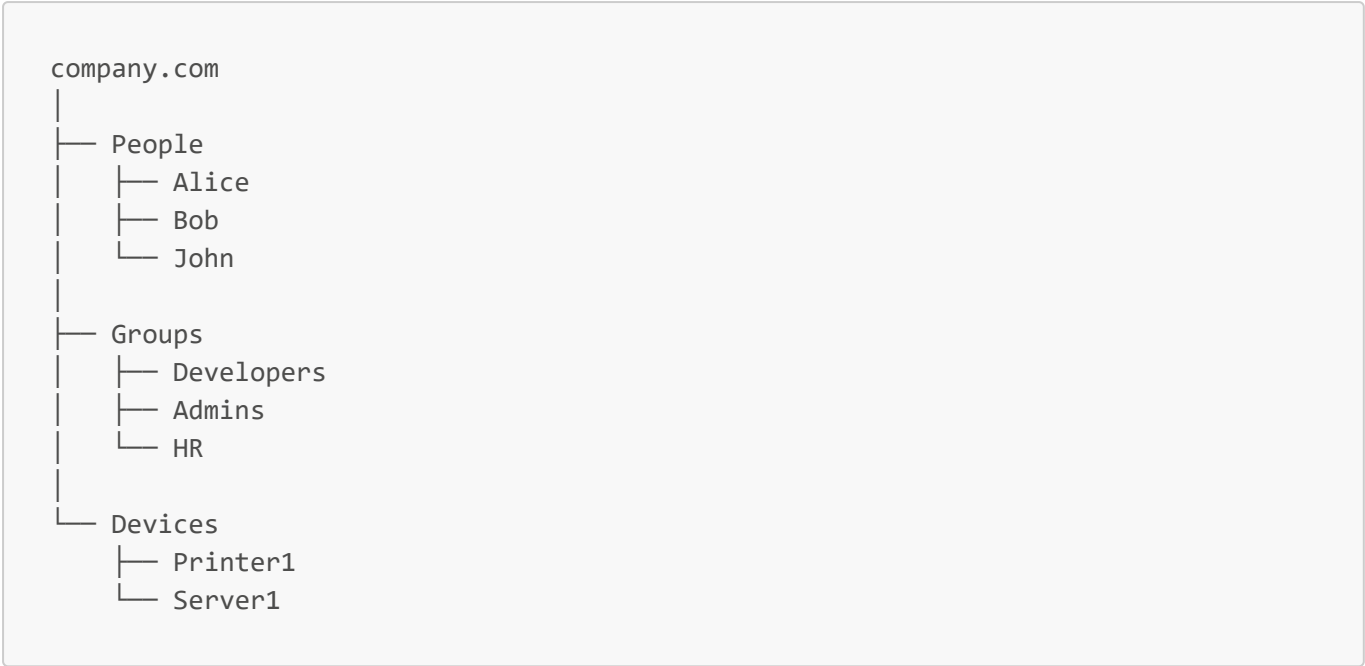
11. Main LDAP Use Cases

Use Case	Example
Centralized Authentication	100 servers + 500 employees all reference ONE LDAP directory instead of separate local accounts on each server
Address Book	Store Name, Email, Phone, Department — searchable by mail clients
Network Resource Management	Track printers, servers, computers, network devices
SSO Infrastructure	One central account usable across many applications (LDAP is part of, not the whole, SSO stack)



12. LDAP Structure — Directory Information Tree (DIT)

LDAP stores data in a **tree structure**, not tables (unlike SQL).



This tree is called the **DIT (Directory Information Tree)**.

Naming Components

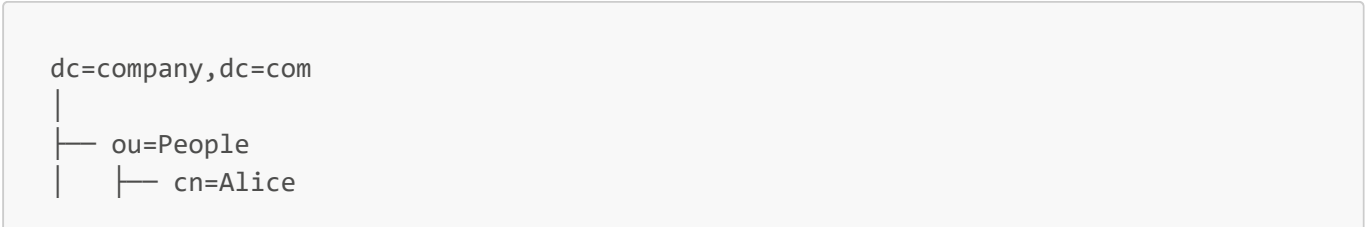
Term	Full Name	Example
DC	Domain Component	dc=company,dc=com (from company.com)
OU	Organizational Unit	ou=People, ou=Groups, ou=IT
CN	Common Name	cn=John Smith
DN	Distinguished Name	Full unique path to an entry

Example Distinguished Name (DN):

```
cn=John Smith,ou=People,dc=company,dc=com
```

```
cn=John Smith → User/object
ou=People      → Organizational unit
dc=company,dc=com → Domain
```

Full Example Tree



```
├── cn=Bob
├── cn=John
├── ou=Groups
│   ├── cn=Admins
│   └── cn=Developers
└── ou=Devices
    ├── cn=Printer1
    └── cn=Server1
```

Exam trap: DN (Distinguished Name) is the **entire unique path** (e.g. `cn=John Smith,ou=People,dc=company,dc=com`), while CN is just **one component** of that path (`cn=John Smith`). Don't confuse DN with CN.

13. LDAP is Read-Heavy

LDAP is optimized mainly for **SEARCH / READ / LOOKUP** operations:

```
"Who is john?"
"What is John's email?"
"Which groups does John belong to?"
```

Writes (create/delete/update) happen far less frequently than reads.

LDAP = Read-heavy and search-optimized

14. LDAP vs SQL Database

Feature	LDAP	SQL Database
Structure	Tree	Tables
Main use	Directory/lookups	General application data
Workload	Read-heavy	Read + write
Query	LDAP filters	SQL
Transactions	Limited	Strong ACID transactions
Authentication	Bind supported	Usually application-managed
Typical data	Users/groups/resources	Orders/payments/products

LDAP is NOT meant to replace a normal SQL database — they serve different purposes (directory lookups vs transactional application data).

15. Replication & Security

Replication

LDAP Server 1 —replication→ LDAP Server 2

Both maintain copies — benefits: high availability, better performance, backup/redundancy. Clients can use either server.

Security

LDAP → protocol (unencrypted by default, port 389)
TLS → protects communication
LDAPS → LDAP over encrypted connection (port 636)

16. LDAP Operations (like CRUD)

LDAP Operation	Similar To	Meaning
Bind	Authentication	Connect and authenticate
Search	Read	Find entries
Add	Create	Create a new entry
Modify	Update	Change an existing entry
Delete	Delete	Remove an entry
Compare	Check	Check an attribute/value
Unbind	Disconnect	Close LDAP connection

Memory trick:

Bind → Login
Search → Read
Add → Create
Modify → Update
Delete → Remove
Unbind → Disconnect

Bind Example

```
Client → Bind request → LDAP Server → Check credentials → Authenticated
```

Example bind identity: `cn=Manager,dc=example,dc=com`

Search Example

```
ldapsearch -x -b "dc=example,dc=com" "(uid=john)"
```

Possible result:

```
dn: uid=john,ou=People,dc=example,dc=com
uid: john
cn: John
mail: john@example.com
```

Add Example

```
ldapadd -x -D "cn=Manager,dc=example,dc=com" -W -f john.ldif
```

Modify Example

```
ldapmodify -x -D "cn=Manager,dc=example,dc=com" -W -f changes.ldif
```

E.g. changing John's department: `IT` → `Security`

Delete Example

```
ldapdelete -x -D "cn=Manager,dc=example,dc=com" -W
"uid=john,ou=People,dc=example,dc=com"
```

17. LDAP Search Filters

Filters describe what to find.

Filter	Meaning
<code>(uid=john)</code>	Find entry where UID is <code>john</code>
<code>(cn=John)</code>	Find common name John

Filter	Meaning
<code>(mail=john@example.com)</code>	Find matching email
<code>(uid=j*)</code>	Wildcard — UID starts with <code>j</code>
<code>(&(objectClass=person)(uid=john))</code>	AND condition — both must match

AND filter breakdown:

```
(&(objectClass=person)(uid=john))  
  ↓           ↓  
objectClass=person  AND  uid=john
```

18. LDAP Security — Authentication Methods

Simple Authentication

```
ldapsearch -x -D "cn=Manager,dc=example,dc=com" -w password
```

`-x` = simple authentication.

Important: Simple authentication should be protected using **TLS**, otherwise credentials aren't safe in transit. **Base64 encoding is NOT encryption** — it's just an encoding scheme, easily reversible.

SASL (Simple Authentication and Security Layer)

Provides advanced authentication mechanisms beyond simple bind:

```
GSSAPI    → Kerberos-based  
DIGEST-MD5  
EXTERNAL
```

```
LDAP Client → SASL/Kerberos → LDAP Server
```

TLS / LDAPS

```
Client → Encrypted LDAP → LDAP Server
```

Port	Use
389	LDAP / STARTTLS possible
636	LDAPS (implicit TLS)

Exam trap: Don't confuse Base64 with encryption. LDAP simple bind sends credentials Base64-*encoded* by default — this is trivially decodable, NOT secure, unless wrapped in TLS/LDAPS.

19. LDAP Password Storage

Common hash formats:

```
{SSHA}
{SHA}
{CRYPT}
{MD5}
{CLEARTEXT}
```

Generate a password hash:

```
slappasswd -s password
```

```
password → slappasswd → {SSHA}...
```

Key rule: LDAP should store a **password hash**, never the original plain-text password.

20. LDAP Client Commands — Quick Reference

Command	Purpose	Example
<code>ldapsearch</code>	Search the directory	<code>ldapsearch -x -b "dc=example,dc=com" "(uid=john)"</code>
<code>ldapadd</code>	Add a new entry	<code>ldapadd -x -D "cn=Manager" -W -f entry.ldif</code>
<code>ldapmodify</code>	Change an existing entry	<code>ldapmodify -x -D "cn=Manager" -W -f changes.ldif</code>
<code>ldapdelete</code>	Delete an entry	<code>ldapdelete -x -D "cn=Manager" -W "uid=john,ou=People"</code>
<code>ldappasswd</code>	Change LDAP password	<code>ldappasswd -x -D "uid=john" -W -S (-W=bind password, -S=new password)</code>

21. SSSD (System Security Services Daemon)

SSSD helps a Linux client connect to centralized identity systems like LDAP.

```
Linux Client → SSSD → LDAP Server → Users/Groups
```

Instead of manually creating `john`, `alice`, `bob` in local `/etc/passwd`, the Linux machine looks them up via SSSD → LDAP.

Main config file: `/etc/sss/sss.conf`

SSSD connects Linux authentication/user lookup to LDAP (or other identity providers).

Full stack:

```
LDAP      → Protocol
OpenLDAP  → Software
slapd     → Server daemon
SSSD      → Common Linux client-side identity daemon
```

```
Linux Client → SSSD → Network → slapd → LDAP Database
```

22. LDAP Schema

A **schema** defines what type of data is allowed in LDAP — rules for objects and attributes.

Example — a user entry may contain: `cn`, `sn`, `uid`, `mail`, `telephoneNumber`.

Schema defines:

```
Which attributes exist?
Which are required vs optional?
What type of value can they contain?
```

Common Schema Files

Location: `/etc/openldap/schema/`

Schema File	Purpose
<code>core.ldif</code>	Basic LDAP objects
<code>cosine.ldif</code>	Common Internet/X.500 attributes
<code>inetorgperson.ldif</code>	User/person objects

Schema File	Purpose
<code>nis.ldif</code>	Unix/NIS attributes
<code>openldap.ldif</code>	OpenLDAP-specific definitions

Importing a schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/cosine.ldif
```

`cosine.ldif` → `ldapadd` → LDAP configuration → Schema becomes available

23. LDIF (LDAP Data Interchange Format)

LDIF is a text format used to create, modify, export, and import LDAP entries.

Example:

```
dn: uid=john,ou=People,dc=example,dc=com
objectClass: inetOrgPerson
uid: john
cn: John Smith
sn: Smith
mail: john@example.com
```

Field	Meaning
<code>dn</code>	Unique location of the entry
<code>objectClass</code>	What type of object it is
<code>uid</code> / <code>cn</code> / <code>sn</code> / <code>mail</code>	Attributes

Adding LDIF data:

```
ldapadd -x -D "cn=Manager,dc=example,dc=com" -W -f john.ldif
```

`john.ldif` → `ldapadd` → LDAP server → John entry created

24. LDAP Utilities

Utility	Purpose
<code>slapcat</code>	Exports LDAP database to LDIF (backup/export)
<code>slapindex</code>	Rebuilds LDAP database indexes (faster searches)
<code>slappasswd</code>	Generates password hashes
<code>ldapvi</code>	Edit LDAP entries in a text-editor style interface

```
slapcat: LDAP database → slapcat → LDIF output
```

GUI LDAP Tools

```
Apache Directory Studio
phpLDAPadmin
LDAP Admin
LDAP Account Manager
```

Provide a graphical browse/manage experience instead of raw CLI commands.

25. NIS vs LDAP — Full Comparison

Feature	NIS	LDAP
Architecture	Old directory service	Modern directory protocol
Data model	Flat maps	Hierarchical tree (DIT)
Security	Weak	TLS/SASL supported
Search	Simple key lookup	Advanced filters
Scalability	Limited	High
Schema	Fixed	Extensible
Modern usage	Legacy	Widely used
Typical structure	<code>passwd.byname</code>	DIT entries
Authentication	Basic	Bind/SASL etc.

```
NIS:
passwd.byname
group.byname
hosts.byname
```

```
LDAP:
dc=example,dc=com
├── ou=People
│   └── John
```

```
(flat maps) |└─ Alice
            └─ ou=Groups
```

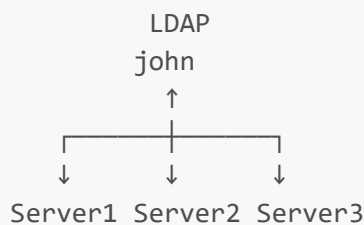
Easy memory:

NIS → Older + flat
LDAP → Modern + hierarchical

26. Complete LDAP Login Example

```
Username: john
Password: ****
  ↓
Linux Login
  ↓
SSSD
  ↓
LDAP Server (slapd)
  ↓
Search: (uid=john)
  ↓
John found
  ↓
Authentication
  ↓
Access granted
```

Instead of managing **john** separately on Server1, Server2, Server3, one LDAP directory serves all:



28. Bonus Note — SSH Key Exchange (Correction)

Important correction: SSH does **not** simply encrypt all session data directly using the server's public/private key pair.

Actual simplified flow:

```
Client
↓
Connects to SSH Server
↓
Server proves identity using its host key
↓
Key exchange happens
↓
Both sides derive shared session keys
↓
Symmetric encryption protects the session
```

```
Public/private key → Authentication/identity + key exchange
Session key       → Actual fast symmetric encryption of SSH traffic
```

Command:

```
ssh user@172.16.140.216
```

Exam trap: Asymmetric (public/private) keys are used for identity verification and key exchange — NOT for encrypting the bulk of the session data. The actual traffic is protected using fast **symmetric** session keys derived during the handshake.